Technische Hochschule Augsburg

Fakultät für Informatik

Bachelorarbeit

zur Erlangung des Grades
Bachelor of Science (B. Sc.) in Informatik

Datenbankleistung bei hoher Last im E-Commerce

erstellt von Anzilal Wahyuasmoro

Anzilal Wahyuasmoro

Dechinger Str. 90A
86165 Augsburg
T +49 176-73885587
anzilal.wahyuasmoro@tha.de

Matrikelnummer:
2125752

Technische Hochschule Augsburg

An der Hochschule 1
D-86161 Augsburg
T +49 821 5586-0
F +49 821 5586-3222
www.tha.de
info@tha.de

Erstprüfer	Prof. Dr. Michael Predeschly
Zweitprüfer	Prof. Matthias Kolonko
Themenvergabe am	19.02.2026
Eingereicht am	22.06.2026
Geheimhaltungsvereinbarung	Nein

Inhaltsverzeichnis

1. Einleitung 1
   - 1.1. Hintergrund und Motivation . . . 1
   - 1.2. Ziel der Arbeit . . . 2
   - 1.3. Verwandte Arbeiten . . . 4
   - 1.4. Aufbau der Arbeit . . . 5

2. Konzeptioneller und methodischer Rahmen 6
   - 2.1. Herleitung des E-Commerce-nahen Lastprofils . . . 6
   - 2.2. Methodische Grundlagen des Datenbankvergleichs . . . 7
     - 2.2.1. Benchmarking, Workloadkontrolle und Bewertungslogik . . . 9
     - 2.2.2. Architekturbezogene Vergleichsdimensionen . . . 10
   - 2.3. Vergleichsdesign und Rahmenbedingungen . . . 13
     - 2.3.1. Messaufbau und Testumgebung . . . 15
     - 2.3.2. Datenmodell, Datengrundlage und E-Commerce-Lastprofil . . . 17
     - 2.3.3. Metriken, Aggregation und Bewertungskriterien . . . 20
     - 2.3.4. Geltungsbereich und Interpretationsannahmen . . . 22
   - 2.4. PostgreSQL als relationales Vergleichssystem . . . 23
   - 2.5. MongoDB als dokumentenorientiertes Vergleichssystem . . . 26

3. Empirische Evaluation 30
   - 3.1. Durchführung und Validierung der Messreihen . . . 30
   - 3.2. Referenzreihe bei 5 bis 100 Usern . . . 32
     - 3.2.1. Operationsbezogene Betrachtung bei 100 Usern . . . 34
     - 3.2.2. Zwischenfazit der Referenzreihe . . . 35
   - 3.3. Replizierter Hochlastbereich bei 500 und 1.000 Usern . . . 35
     - 3.3.1. Durchsatz und Median-Latenz . . . 36
     - 3.3.2. Tail-Latenzen und Fehlerrate . . . 37
     - 3.3.3. Zwischenfazit des replizierten Hochlastbereichs . . . 40
   - 3.4. Explorative Grenzlast bei 2.500 und 5.000 Usern . . . 40
     - 3.4.1. Durchsatz und Latenzverhalten in der Grenzlast . . . 41
     - 3.4.2. Fehlerrate und Stabilität in der Grenzlast . . . 42
     - 3.4.3. Zwischenfazit der explorativen Grenzlast . . . 42
   - 3.5. Datenmengen-Sensitivitätsanalyse . . . 43
     - 3.5.1. Durchsatz und Median-Latenz bei größerem Produktkatalog . . . 43
     - 3.5.2. Tail-Latenzen bei größerem Produktkatalog . . . 44

3.5.3. Stabilität und Zwischenfazit der Datenmengen-Sensitivitätsanalyse . . 46
3.6. Zusammenführende Ergebnisbewertung 47

4. Schlussbetrachtung 49
4.1. Beantwortung der Forschungsfragen 49
4.2. Methodische Einordnung und Limitationen 52
4.3. Ausblick 53
4.4. Fazit 55

A. Ergänzende Versuchsdokumentation 56
A.1. Standardisierte Versuchspipeline 56
A.2. Digitale Begleitmaterialien 59

Literaturverzeichnis 64

1. Einleitung

E-Commerce-Systeme sind auf eine leistungsfähige und zuverlässige Datenhaltung angewiesen. Besonders in zeitlich begrenzten Lastsituationen können viele parallele Nutzerinteraktionen zu einer erheblichen Belastung der zugrunde liegenden Datenbanksysteme führen. Die vorliegende Arbeit untersucht deshalb, wie sich zwei unterschiedliche Datenbanksysteme unter typischen Belastungen eines Online-Shops verhalten.

1.1. Hintergrund und Motivation

Zeitlich begrenzte Shopping-Events wie Black Friday, Cyber Monday und die Cyber Week führen im E-Commerce regelmäßig zu hohen, kurzzeitigen wirtschaftlichen und technischen Belastungssituationen. Für den US-Markt berichtet Adobe für den Cyber Monday 2025 ein Online-Umsatzvolumen von 14,25 Milliarden US-Dollar. Für die fünf Tage von Thanksgiving bis Cyber Monday 2025 weist Adobe 44,2 Milliarden US-Dollar Online-Umsatz aus [1]. Ergänzend berichtet Salesforce für Black Friday 2025 ein globales Umsatzvolumen von 79 Milliarden US-Dollar sowie 18 Milliarden US-Dollar in den USA [2]. Die genannten Marktberichte dienen in dieser Arbeit als aktuelle Motivationsbeispiele für die wirtschaftliche Relevanz zeitlich konzentrierter E-Commerce-Lastsituationen. Die spätere technische Bewertung stützt sich auf die im Rahmen dieser Arbeit erhobenen Messwerte.

Für datenbankgestützte Online-Shops sind solche Lastspitzen besonders bedeutsam, weil typische Nutzerinteraktionen fachliche Prozesse auslösen, die in der Anwendung häufig mit Datenbankzugriffen verbunden sind. Dazu zählen insbesondere Suchanfragen, Produktübersichten und Produktdetailseiten sowie transaktionsnahe Vorgänge im Warenkorb- und Bestellprozess. Aktuelle E-Commerce-Messmodelle erfassen vergleichbare Schritte entlang des Kaufprozesses, darunter Produktansichten, Warenkorbaktionen, den Beginn des Checkouts und Kaufvorgänge [3].

Aus Datenbanksicht lassen sich diese Interaktionen auf unterschiedliche Zugriffsmuster zurückführen. Produktbezogene Vorgänge wie Suche, Produktanzeige, Preis- oder Verfügbarkeitsabfragen führen überwiegend zu Lesezugriffen. Warenkorb- und checkoutnahe Vorgänge können hingegen Schreib- und Aktualisierungsoperationen auslösen, etwa beim Ändern von Warenkorbeinträgen, beim Anlegen einer Bestellung oder bei Bestandsaktualisierungen. Diese Einordnung knüpft an E-Commerce-nahe Workloadmodelle an, die produkt-, warenkorb- und bestellbezogene Vorgänge als unterschiedliche Bestandteile eines Anwendungsszenarios betrachten [3, 4, 5]. Mit steigender

Anzahl paralleler Nutzeranfragen erhöht sich damit auch die gleichzeitige Belastung der Datenbank [6, S. 75, S. 288–290].

Vor diesem Hintergrund untersucht die vorliegende Arbeit die Datenbankleistung unter hoher Last im E-Commerce. Damit ergibt sich die Grundlage für die Zielsetzung der Arbeit: die kontrollierte Untersuchung von Datenbankleistung unter einem E-Commerce-nahen Belastungsszenario.

1.2. Ziel der Arbeit

Ausgehend von den in Abschnitt 1.1 dargestellten Lastspitzen im E-Commerce stellt sich die Frage, wie sich Datenbanksysteme unter solchen Belastungssituationen konkret verhalten. Mit steigender Anzahl paralleler Nutzeranfragen nehmen die gleichzeitigen Lese- und Schreibzugriffe auf die Datenbank zu. Dadurch kann die Datenbank in datenbankgestützten Online-Shops zu einem zentralen Engpass werden, insbesondere wenn unter hoher Parallelität Ressourcensättigung, Warteschlangenbildung oder steigende Antwortzeiten auftreten [6, S. 75, S. 288–290].

Für die Bewertung solcher Lastsituationen ist jedoch nicht allein die maximale Verarbeitungsgeschwindigkeit relevant. Ebenso entscheidend ist, wie stabil und zuverlässig ein System unter wachsender Last arbeitet, etwa gemessen über Latenz, Fehlerrate und Durchsatz [7, 8]. In Anlehnung an diese Bewertungsgrößen wird Leistungsdegradation in dieser Arbeit als Verschlechterung des gemessenen Systemverhaltens bei zunehmender Last verstanden. Sie kann sich beispielsweise durch stagnierenden oder sinkenden Durchsatz, steigende Antwortzeiten, höhere Fehlerraten oder zunehmende Wartezeiten zeigen. Für die Beurteilung der Datenbankleistung ist daher nicht nur die absolute Geschwindigkeit maßgeblich, sondern auch das Verhalten des Systems unter fortschreitender Last.

Hinzu kommt, dass Datenbanksysteme je nach Datenmodell, Umgang mit gleichzeitigen Zugriffen und Transaktionen, Anfrageformen und konkretem Workload unterschiedlich auf vergleichbare Zugriffsmuster reagieren können [9, 10]. Für ein E-Commerce-nahes Lastprofil ist daher nicht unmittelbar ersichtlich, wie sich unterschiedliche Datenbankmanagementsysteme unter denselben Belastungsbedingungen hinsichtlich Leistung und Stabilität verhalten. Erforderlich ist deshalb ein kontrollierter und nachvollziehbarer Vergleich unter einheitlichen Testbedingungen.

Ziel dieser Arbeit ist es daher, die Datenbankleistung unter hoher Last im E-Commerce empirisch zu untersuchen. Dafür wird ein E-Commerce-nahes Lastprofil verwendet. Das Lastprofil beschreibt in dieser Arbeit den fachlichen Mix an Operationen, den ein simulierter Nutzer während eines Lasttests ausführt. Es geht dabei also nicht nur um eine bestimmte Anzahl gleichzeitiger Nutzer, sondern auch darum, welche typischen Online-Shop-Aktionen unter dieser Last ausgeführt werden.

Das Profil bildet typische datenbanknahe Vorgänge eines Online-Shops ab: Produkte werden gesucht und gelesen, Preise und Bestände werden abgefragt, Warenkörbe werden gelesen oder verändert und in einem checkoutnahen Vorgang wird eine Bestellung erzeugt.

Tabelle 1.1 gibt einen kompakten Überblick über diese fachlichen Operationen. Die konkrete technische Umsetzung des Lastprofils wird in Kapitel 2 beschrieben.

Tabelle 1.1.: Überblick über die untersuchten E-Commerce-nahen Operationen

Operation	**Abgebildeter Vorgang im Online-Shop**
Produktliste / Produktsuche	Produkte nach Kategorie und Preisbereich abrufen
Produktdetail	Einzelnes Produkt mit Preis- und Bestandsinformationen lesen
Preis- und Bestandsabfrage	Verfügbarkeit und Preis eines Produktes prüfen
Warenkorb lesen	Aktiven Warenkorb eines Kunden abrufen
Warenkorb ändern	Produkt in einen Warenkorb einfügen oder Menge erhöhen
Checkout-nahe Transaktion	Bestellung anlegen, Bestand reduzieren und Warenkorb abschließen

Als konkrete Vergleichssysteme werden PostgreSQL und MongoDB ausgewählt. Beide Systeme sind in der Praxis verbreitet und repräsentieren unterschiedliche Datenbankmodelle [11]. PostgreSQL steht in dieser Arbeit als relationales Datenbankmanagementsystem für ein tabellenbasiertes Modell mit ausgeprägter Transaktionsunterstützung. MongoDB steht als dokumentenorientiertes Datenbankmanagementsystem für ein anderes Datenmodell und andere Entwurfsentscheidungen bei der Abbildung zusammengehöriger Daten. Die Einordnung über das DB-Engines-Ranking begründet die Auswahl verbreiteter Vergleichssysteme; die Leistungsbewertung erfolgt anschließend ausschließlich anhand der in dieser Arbeit erhobenen Messwerte.

Auf dieser Grundlage werden PostgreSQL und MongoDB in mehreren Auswertungsbereichen verglichen: zunächst in einem kontrollierten Referenzbereich, anschließend unter höherer Last, ergänzend in einem explorativen Grenzlastbereich sowie in einer Datenmengen-Sensitivitätsanalyse. Die konkreten Laststufen, Wiederholungszahlen und Versuchskonfigurationen werden im methodischen Rahmen der Arbeit in Abschnitt 2.3 beschrieben.

Der Beitrag der Arbeit besteht in einem strukturierten und nachvollziehbaren Vergleich zweier konkreter Systeme unter einem definierten E-Commerce-nahen Lastprofil. Durch die Verbindung eines hergeleiteten Lastprofils, kontrollierter Testbedingungen und einer systematischen Auswertung werden Unterschiede im Leistungs- und Stabilitätsverhalten beider Systeme sichtbar gemacht.

Daraus ergeben sich für die Arbeit die folgenden Forschungsfragen:

- **F1:** Wie verhalten sich PostgreSQL und MongoDB in einem kontrollierten Referenzbereich hinsichtlich Durchsatz, Latenz und Fehlerrate?
- **F2:** Wie verändern sich Durchsatz, Latenzverteilung und Fehlerrate beider Systeme bei steigender paralleler Last bis in einen stark belasteten Grenzbereich?
- **F3:** Wie wirkt sich eine Vergrößerung des Produktkatalogs bei konstanter Last auf das Leistungs- und Latenzverhalten von PostgreSQL und MongoDB aus?
- **F4:** Wie lassen sich die beobachteten Unterschiede im dokumentierten E-Commerce-nahen Versuchsaufbau einordnen?

1.3. Verwandte Arbeiten

Verwandte Arbeiten zur Untersuchung von Datenbankperformance zeigen, dass Leistungsvergleiche zwischen Datenbankmanagementsystemen nur dann aussagekräftig sind, wenn Workload, Datengrundlage, Testumgebung und Bewertungsmetriken nachvollziehbar beschrieben werden [9]. Taipalus untersucht in einer systematischen Literaturübersicht bestehende Performancevergleiche von Datenbankmanagementsystemen und zeigt, dass sich Studien unter anderem hinsichtlich Workload, Messmethodik, Datenmodell und Reproduzierbarkeit deutlich unterscheiden [9]. Für die vorliegende Arbeit ist dies insofern relevant, als der Vergleich von PostgreSQL und MongoDB als szenariospezifischer Datenbankvergleich mit definiertem Lastprofil, dokumentierter Datengrundlage und kontrollierten Rahmenbedingungen angelegt wird.

Auch Benchmarking-Frameworks verdeutlichen die Bedeutung definierter Workloads und kontrollierter Messbedingungen. BenchBase bietet einen erweiterbaren Rahmen zur Durchführung vergleichbarer Datenbank-Benchmarks und ist auf OLTP- sowie weborientierte Workloads ausgerichtet [12]. Das Framework unterstützt unter anderem variable Last- und Transaktionsmischungen sowie die Erfassung transaktionsbezogener Latenz- und Durchsatzwerte [13]. Solche Ansätze sind für diese Arbeit methodisch relevant, weil sie zeigen, dass Datenbankleistung nicht isoliert betrachtet wird, sondern immer im Zusammenhang mit dem zugrunde liegenden Workload und der gewählten Bewertungslogik steht.

Für E-Commerce-nahe Anwendungsszenarien existieren zudem Arbeiten, die moderne Workloadmodelle anhand fachlicher Abläufe wie Produktzugriffen, Warenkorbaktionen, Checkout-Prozessen, Bestandsinformationen und Bestellverarbeitung beschreiben [4, 5].

Laigner et al. schlagen mit dem Online Marketplace Benchmark einen Benchmark für Datenmanagement in Microservice-basierten Marktplatzsystemen vor [4]. Der Benchmark adressiert dabei zentrale Datenmanagement-Herausforderungen solcher Anwendungen, unter anderem Transaktionsverarbeitung, Abfrageverarbeitung, Ereignisverarbeitung, Integritätsbedingungen und Replikation [4]. Für die vorliegende Arbeit ist dies relevant, weil E-Commerce-nahe Szenarien nicht nur einzelne Leseabfragen, sondern eine Kombination aus produkt-, warenkorb- und checkoutbezogenen Zugriffsmustern umfassen.

Van Landuyt et al. betrachten E-Commerce-Szenarien speziell im Kontext dokumentenorientierter Datenbanksysteme, die in der Literatur häufig auch als Document Stores bezeichnet werden [5]. Im Mittelpunkt stehen dabei Performance-Trade-offs, die sich aus unterschiedlichen Modellierungsentscheidungen und Workload-Eigenschaften ergeben [5]. Diese Arbeiten sind für die vorliegende Arbeit relevant, weil sie zeigen, dass E-Commerce-nahe Lastprofile aus mehreren fachlich unterschiedlichen Zugriffsmustern bestehen und nicht auf einzelne isolierte Datenbankoperationen reduziert werden können.

Direkte Vergleiche zwischen PostgreSQL und MongoDB liegen ebenfalls vor, beziehen sich jedoch nicht zwingend auf E-Commerce-nahe Lastprofile [10]. Makris et al. vergleichen MongoDB und PostgreSQL in einem Anwendungskontext mit räumlichen und zeitlichen Daten und zeigen dabei, dass Datenmodell, Indexierung und konkrete Anfrageformen einen erheblichen Einfluss auf die gemessene Performance haben können [10]. Damit wird deutlich, dass Ergebnisse solcher Systemvergleiche stark vom jeweiligen Anwendungsszenario und Workload abhängen.

1.4. Aufbau der Arbeit

Kapitel 2 beschreibt den konzeptionellen und methodischen Rahmen der Arbeit. Zunächst wird das E-Commerce-nahe Lastprofil hergeleitet. Anschließend werden die beiden Vergleichssysteme, die Datengrundlage, die lokale Testumgebung, die Versuchskonfigurationen und die standardisierte Versuchspipeline beschrieben.

Kapitel 3 enthält die empirische Evaluation. Die Ergebnisdarstellung folgt der Struktur der Untersuchung: Referenzreihe, replizierter Hochlastbereich, explorative Grenzlast und Datenmengen-Sensitivitätsanalyse. Die Messwerte werden anhand von Durchsatz, Median-Latenz, p95- und p99-Latenzen sowie Fehlerraten ausgewertet und abschnittsweise interpretiert.

Kapitel 4 fasst die Ergebnisse zusammen, beantwortet die Forschungsfragen und ordnet die methodischen Grenzen der Untersuchung ein. Abschließend werden mögliche Erweiterungen des Versuchsaufbaus und Ansatzpunkte für weiterführende Arbeiten dargestellt.

2. Konzeptioneller und methodischer Rahmen

Dieses Kapitel legt den konzeptionellen und methodischen Rahmen für die empirische Untersuchung fest. Ausgehend von der in Abschnitt 1.2 formulierten Zielsetzung wird zunächst eingeordnet, wie E-Commerce-spezifische Lastspitzen in ein für die Untersuchung nutzbares Lastprofil überführt werden können. Darauf aufbauend wird dargestellt, wie Datenbankperformance anhand definierter Workloads, kontrollierter Testbedingungen und geeigneter Bewertungsmetriken untersucht werden kann.

Anschließend werden das Vergleichsdesign und die Rahmenbedingungen der Arbeit konkretisiert. Dazu gehören insbesondere der Messaufbau, die Testumgebung, das Datenmodell, die Definition des E-Commerce-nahen Lastprofils sowie die verwendeten Metriken und Bewertungskriterien. Ergänzend werden die für den Vergleich relevanten Eigenschaften von PostgreSQL und MongoDB eingeordnet, damit die späteren Messergebnisse im Kontext der jeweiligen Architekturprinzipien interpretiert werden können.

2.1. Herleitung des E-Commerce-nahen Lastprofils

Die in Abschnitt 1.1 beschriebenen stark frequentierten Verkaufsereignisse begründen, weshalb eine hohe Zahl gleichzeitiger Nutzerinteraktionen für E-Commerce-Systeme technisch relevant ist. Aus Umsatz- oder Internet-Traffic-Werten lässt sich jedoch noch nicht unmittelbar ableiten, welche Datenbankoperationen in einem Lasttest ausgeführt werden sollen. Für den empirischen Vergleich muss deshalb zunächst bestimmt werden, welche fachlichen Interaktionen eines Online-Shops datenbankrelevante Zugriffsmuster auslösen und wie diese in ein kontrolliert ausführbares Lastprofil überführt werden.

E-Commerce-Eventmodelle unterscheiden entlang des Kaufprozesses unter anderem Produktlisten- und Produktdetailansichten, Warenkorbaktionen, den Beginn des Checkout-Prozesses sowie abgeschlossene Käufe [3]. Auch E-Commerce- und Marketplace-orientierte Benchmarks bilden nicht nur eine einzelne Abfrage, sondern eine Kombination aus Produktzugriffen, Warenkorboperationen, Bestellverarbeitung und transaktionsbezogenen Vorgängen ab [4, 5]. Diese Prozessschritte bilden den fachlichen Ausgangspunkt für das in dieser Arbeit verwendete Lastprofil.

Für die Untersuchung werden die fachlichen Interaktionen in drei datenbanknahe Zugriffsklassen überführt. Die erste Klasse umfasst produktbezogene Lesezugriffe, beispielsweise das Filtern und Sortieren von Produktlisten, den Abruf eines einzelnen Produkts sowie Preis- und Bestandsabfragen. Die zweite Klasse enthält Warenkorbzugriffe, bei denen Warenkörbe gelesen oder durch das Hinzufügen beziehungsweise Aktualisieren von Positionen verändert werden. Die dritte Klasse bildet den vereinfachten Bestellabschluss ab. Dabei werden zusammengehörige Lese- und Schreiboperationen auf Warenkorb-, Bestell- und Bestandsdaten innerhalb eines transaktionsnahen Ablaufs ausgeführt. Diese Einteilung stellt eine Operationalisierung für den vorliegenden Vergleich dar. Sie verbindet fachlich erkennbare Schritte des Kaufprozesses mit solchen Zugriffsmustern, die auf Datenbankebene gezielt ausgeführt und gemessen werden können.

Unter einem Lastprofil wird in dieser Arbeit die festgelegte Kombination aus fachlichen Operationen und den Parametern ihrer Ausführung verstanden. Dazu gehören insbesondere

- die Menge der ausgeführten Operationstypen,
- deren relative Auswahlwahrscheinlichkeiten,
- die Wartezeit zwischen zwei Operationen eines simulierten Nutzers,
- die Zahl gleichzeitig aktiver Nutzer,
- die Geschwindigkeit, mit der diese Nutzer gestartet werden,
- die Regeln zur Auswahl der bearbeiteten Datensätze sowie
- die Dauer eines Messlaufs.

Die Anzahl der gleichzeitig simulierten Nutzer bildet dabei die primär kontrollierte Lastgröße. Die tatsächlich erreichte Anzahl von Requests pro Sekunde wird nicht vorgegeben, sondern ergibt sich aus den Antwortzeiten der Datenbanksysteme, den ausgeführten Operationen und den festgelegten Wartezeiten. Der Durchsatz ist daher eine Ergebnisgröße und nicht mit der eingestellten Nutzerzahl gleichzusetzen.

Aus dem Untersuchungsziel werden folgende Anforderungen an das Lastprofil abgeleitet:

Die Anforderungen LP1 bis LP6 aus Tabelle 2.1 bilden die Grundlage für die konkrete Parametrisierung des Lastprofils. Die Auswahl der einzelnen Operationen, ihre Gewichtungen und die zugrunde liegende Datengrundlage werden in Abschnitt 2.3.2 festgelegt. Die technische Ausführung und Laststeuerung werden in Abschnitt 2.3.1 beschrieben. Die daraus abgeleiteten Bewertungsgrößen folgen in Abschnitt 2.3.3.

2.2. Methodische Grundlagen des Datenbankvergleichs

Ein gemessener Leistungswert ist keine vom Untersuchungsaufbau unabhängige Eigenschaft eines Datenbanksystems. Durchsatz, Antwortzeiten und Fehlerverhalten werden unter anderem durch den ausgeführten Workload, die Datengrundlage, die Systemkonfiguration, die verfügbare Hardware und das Auswertungsverfahren beeinflusst. Performancevergleiche sind deshalb nur dann nachvollziehbar, wenn diese Rahmenbedingungen transparent beschrieben und kontrolliert werden

Tabelle 2.1.: Anforderungen an das E-Commerce-nahe Lastprofil

Nr.	Anforderung	Begründung
LP1	Fachliche Abdeckung zentraler Prozessschritte	Das Lastprofil soll Produktzugriffe, Warenkorboperationen und einen vereinfachten Bestellabschluss enthalten, damit unterschiedliche Phasen des E-Commerce-Kaufprozesses berücksichtigt werden.
LP2	Kombination verschiedener Zugriffsklassen	Neben häufigen Lesezugriffen müssen auch schreibende, aktualisierende und transaktionsnahe Vorgänge enthalten sein, da diese andere Anforderungen an Nebenläufigkeit und Datenkonsistenz stellen.
LP3	Fachliche Vergleichbarkeit	PostgreSQL und MongoDB sollen jeweils dieselben fachlichen Ergebnisse erzeugen. Die technische Umsetzung darf sich entsprechend dem relationalen beziehungsweise dokumentenorientierten Datenmodell unterscheiden.
LP4	Kontrollierbare Lastintensität	Die Parallelität muss über definierte Nutzerzahlen, Spawn-Raten, Wartezeiten und Laufzeiten schrittweise verändert werden können.
LP5	Wiederholbare Ausführung	Operationen, Gewichtungen und Auswahlregeln müssen für die zu vergleichenden Läufe fest vorgegeben sein, damit Unterschiede nicht auf wechselnde Versuchsparameter zurückzuführen sind.
LP6	Getrennte Messbarkeit der Operationen	Jeder Operationstyp muss einzeln an das Lasttestwerkzeug gemeldet werden, damit sowohl aggregierte als auch operationsbezogene Latenz-, Durchsatz- und Fehlerwerte ausgewertet werden können.

[9]. Direkte Vergleiche zwischen PostgreSQL und MongoDB zeigen zudem, dass Datenmodell, Indexierung und konkrete Anfrageformen einen erheblichen Einfluss auf die Messergebnisse haben können [10].

Aufbauend auf dem in Abschnitt 2.1 hergeleiteten Lastprofil werden im Folgenden die methodischen Grundlagen des Systemvergleichs konkretisiert. Abschnitt 2.2.1 behandelt zunächst Benchmarking, Workloadkontrolle und Bewertungslogik. Anschließend werden die für die spätere Interpretation relevanten architekturbezogenen Eigenschaften der beiden Datenbanksysteme eingeordnet.

2.2.1. Benchmarking, Workloadkontrolle und Bewertungslogik

Benchmarking.

Benchmarking bezeichnet im Rahmen dieser Arbeit die geplante und wiederholbare Messung des Systemverhaltens unter zuvor festgelegten Bedingungen. Ein belastbarer Vergleich setzt voraus, dass nicht nur die Ergebnisse, sondern auch der verwendete Datenbestand, die ausgeführten Operationen, die Lastparameter, die Systemkonfiguration und das Auswertungsverfahren dokumentiert werden. Taipalus zeigt, dass sich bestehende Datenbankvergleiche gerade hinsichtlich Workload, Datenmodell, Messmethodik und Reproduzierbarkeit deutlich unterscheiden [9]. Die Aussagekraft eines Ergebnisses ist daher an den jeweiligen Versuchsaufbau gebunden.

Workloadkontrolle.

Das in Abschnitt 2.1 definierte Lastprofil legt die fachlichen Operationen und die Parameter ihrer Ausführung fest. Für einen direkten Systemvergleich müssen diese Parameter innerhalb einer Versuchsreihe möglichst konstant gehalten werden. Dazu gehören insbesondere die fachlichen Operationen, deren Gewichtungen, die Wartezeiten, die Nutzerzahl, die Spawn-Rate, die Laufzeit und die zugrunde liegende Datengrundlage.

Fachliche Gleichheit bedeutet dabei nicht, dass PostgreSQL und MongoDB identische technische Anweisungen ausführen. PostgreSQL verarbeitet die Zugriffsmuster über relationale Tabellen und SQL-Anweisungen, während MongoDB Collections und Dokumentoperationen verwendet. Vergleichbar sind die Operationen dann, wenn sie denselben fachlichen Zweck erfüllen und entsprechende Daten lesen oder verändern. Die systemspezifische technische Umsetzung bleibt Bestandteil des Vergleichs und muss deshalb transparent dokumentiert werden.

Die Vergleichbarkeit gilt jeweils innerhalb einer definierten Versuchsreihe. Änderungen zwischen der Referenzreihe, dem replizierten Hochlastbereich, der explorativen Grenzlast und der Datenmengen-Sensitivitätsanalyse werden als methodische Änderungen ausgewiesen. Dies betrifft insbesondere die erhöhte Verbindungsgrenze von PostgreSQL und die Verwendung eines gemeinsam genutzten MongoDB-Clients im Hochlastbereich.

Bewertungslogik.

Die Bewertung eines Datenbanksystems darf nicht auf einer einzelnen Kennzahl beruhen. Ein hoher Durchsatz kann beispielsweise gleichzeitig mit stark steigenden Antwortzeiten auftreten. Ebenso bedeutet eine Fehlerrate von null nicht automatisch, dass die Verarbeitung für einen praktischen E-Commerce-Einsatz ausreichend schnell erfolgt.

Zur Bewertung werden deshalb Durchsatz, Median-Latenz, p95-Latenz, p99-Latenz und Fehlerrate gemeinsam betrachtet. Der Median beschreibt das typische Antwortzeitverhalten. p95 und p99 machen den oberen Bereich der Latenzverteilung sichtbar, in dem langsamere erfolgreiche Operationen liegen.

Die Betrachtung solcher Tail-Latenzen ist für Online-Systeme relevant, weil ein kleiner Anteil besonders langsamer Anfragen die wahrgenommene Systemleistung beeinflussen kann [14]. Latenz und Fehlerrate werden auch im Site Reliability Engineering als quantitative Indikatoren zur Beurteilung des Systemverhaltens verwendet [7, 8].

Stabilität wird in dieser Arbeit daher nicht als einzelne, unmittelbar gemessene Kennzahl verstanden. Sie wird anhand des gemeinsamen Verlaufs von erfolgreichem Durchsatz, Latenzverteilung und endgültiger Fehlerrate eingeordnet. Eine Degradation liegt insbesondere dann nahe, wenn eine höhere Parallelität nicht zu zusätzlichem Durchsatz führt, während Median- oder Tail-Latenzen stark steigen oder endgültig fehlgeschlagene Operationen auftreten.

Aus diesen Grundlagen ergeben sich die in Tabelle 2.2 zusammengefassten Kriterien für den empirischen Datenbankvergleich.

Die Kriterien VK1 bis VK7 bilden die Grundlage für das nachfolgend beschriebene Vergleichsdesign. Die konkrete technische Umsetzung, Datengenerierung und Parametrisierung der Versuchsreihen werden in den folgenden Abschnitten festgelegt. Die genaue Berechnung und Aggregation der Metriken wird im Abschnitt zu den Bewertungskriterien beschrieben.

2.2.2. Architekturbezogene Vergleichsdimensionen

Die in Abschnitt 2.2.1 definierten Vergleichskriterien legen fest, unter welchen Bedingungen die Messungen durchgeführt und bewertet werden. Für die spätere Interpretation ist zusätzlich zu berücksichtigen, dass PostgreSQL und MongoDB dieselben fachlichen Operationen mit unterschiedlichen Datenmodellen, Abfrageformen und Mechanismen zur Verarbeitung paralleler Zugriffe umsetzen.

Aus solchen Unterschieden kann nicht unmittelbar eine Ursache für einen gemessenen Leistungswert abgeleitet werden. Sie bilden jedoch technische Vergleichsdimensionen, anhand derer beobachtete Durchsatz-, Latenz- und Fehlerverläufe eingeordnet werden können. Für die vorliegende Untersuchung werden die in Tabelle 2.3 zusammengefassten Dimensionen betrachtet.

Tabelle 2.2.: Methodische Kriterien des Datenbankvergleichs

Nr.	Kriterium	Konsequenz für die Untersuchung
VK1	Fachliche Äquivalenz	PostgreSQL und MongoDB führen dieselben fachlichen E-Commerce-Operationen aus. Die technische Umsetzung erfolgt systemspezifisch.
VK2	Kontrollierte Bedingungen	Innerhalb einer Versuchsreihe werden Datengrundlage, Nutzerzahl, Spawn-Rate, Laufzeit, Lastprofil und Ressourcenlimits für beide Systeme möglichst einheitlich festgelegt.
VK3	Transparente Konfigurationsunterschiede	Systemspezifische Einstellungen und Änderungen zwischen den Versuchsreihen werden dokumentiert und bei der Interpretation berücksichtigt.
VK4	Reproduzierbarer Ausgangszustand	Die Testdaten werden deterministisch erzeugt. Vor jedem Messlauf wird der definierte Ausgangsdatenbestand erneut geladen, damit vorangegangene Schreiboperationen spätere Läufe nicht beeinflussen.
VK5	Wiederholung und Einordnung der Messreihen	Zentrale Vergleichsbereiche werden wiederholt gemessen. Nicht wiederholte Grenzlastmessungen werden ausdrücklich als explorativ gekennzeichnet.
VK6	Mehrdimensionale Bewertung	Durchsatz, Median, p95, p99 und Fehlerrate werden gemeinsam ausgewertet. Keine einzelne Metrik dient allein als Nachweis eines günstigen oder ungünstigen Systemverhaltens.
VK7	Begrenzte Interpretation	Messergebnisse werden von plausiblen Erklärungen und nicht nachgewiesenen Ursachen getrennt. Aussagen gelten für den dokumentierten Versuchsaufbau und die konkrete Implementierung.

Tabelle 2.3.: Architekturbezogene Dimensionen der Ergebnisinterpretation

Nr.	Vergleichsdimension	Bedeutung für die Untersuchung
I1	Datenorganisation und Zugriffspfade	PostgreSQL verteilt fachliche Entitäten auf relationale Tabellen und stellt Zusammenhänge über Schlüsselbeziehungen und Joins her. MongoDB speichert Daten in Dokumenten. Zusammengehörige Informationen können innerhalb eines Dokuments eingebettet oder über Kennungen referenziert werden. Dadurch unterscheiden sich die für dieselbe fachliche Operation erforderlichen Datenzugriffe.
I2	Indexierung und Anfrageformen	Beide Systeme verwenden systemspezifische Indexstrukturen und Anfrageformen. Die gemessene Leistung hängt daher nicht allein vom Datenbanksystem, sondern auch von der konkreten Abfrage und den dafür verfügbaren Indexen ab [10].
I3	Nebenläufigkeits- und Transaktionsverarbeitung	Das Lastprofil enthält parallele Lese-, Schreib- und Checkout-Operationen. PostgreSQL verwendet unter anderem Multi-Version Concurrency Control und explizite Zeilensperren. MongoDB verwendet dokumentbezogene Aktualisierungen und bei der Checkout-Operation eine Transaktion über mehrere Dokumente. Konflikte und Wartezeiten können sich daher systemspezifisch unterscheiden.
I4	Verbindungsverwaltung	Die Datenbanksysteme werden über ihre jeweiligen Python-Treiber angesprochen. Die Anzahl und Wiederverwendung der Datenbankverbindungen beeinflussen, wie gleichzeitig simulierte Nutzer auf das System zugreifen. Änderungen der Clientstrategie zwischen den Versuchsreihen müssen deshalb dokumentiert und bei deren Vergleich berücksichtigt werden.

Die Dimension I1 betrifft die strukturelle Abbildung des fachlichen E-Commerce-Modells. PostgreSQL verwendet ein relationales Modell, in dem beispielsweise Produkt-, Bestands-, Warenkorb- und Bestelldaten in getrennten Tabellen gespeichert werden. MongoDB ermöglicht dagegen, häufig gemeinsam verwendete Informationen innerhalb eines Dokuments abzulegen. Dieser Vorgang wird als Einbettung beziehungsweise Embedding bezeichnet. Beim Referencing bleiben Informationen in getrennten Dokumenten und werden über eine Kennung miteinander verbunden. Die Wahl zwischen beiden Formen soll sich an den vorgesehenen Zugriffsmustern orientieren [15].

Die Dimension I3 beschreibt die Verarbeitung gleichzeitig ausgeführter Operationen. PostgreSQL verwendet Multi-Version Concurrency Control, wodurch Transaktionen jeweils eine konsistente Sicht auf die Daten erhalten [16]. Bei Operationen, die gemeinsame Datensätze verändern, können zusätzlich explizite Sperren eingesetzt werden. MongoDB führt Änderungen an einem einzelnen Dokument atomar aus; für den in dieser Arbeit modellierten Checkout werden jedoch mehrere Dokumente und Collections gemeinsam verändert. Dafür wird eine Transaktion verwendet. Die konkrete Umsetzung der Checkout-Abläufe wird in den späteren systemspezifischen Abschnitten beschrieben.

Die Verbindungsverwaltung nach I4 ist insbesondere für die unterschiedlichen Versuchsreihen relevant. In der Referenzreihe verwendet jeder simulierte Nutzer jeweils eine eigene PostgreSQL-Verbindung beziehungsweise einen eigenen MongoDB-Client. Für die MongoDB-Hochlast-, Grenzlast- und Sensitivitätsmessungen wird dagegen ein gemeinsam initialisierter MongoDB-Client mit Connection Pool verwendet. PostgreSQL verwendet weiterhin eine eigene Verbindung pro simuliertem Nutzer. Diese unterschiedlichen Clientstrategien sind Bestandteil des realisierten Messaufbaus und werden bei der Interpretation der Ergebnisse ausdrücklich berücksichtigt.

Die Dimensionen I1 bis I4 dienen ausschließlich als Interpretationsrahmen. Ein gemessener Unterschied darf nicht ohne zusätzliche Nachweise einer einzelnen Dimension als eindeutige Ursache zugeschrieben werden. Die empirische Auswertung trennt daher zwischen einem beobachteten Messergebnis, einer durch die Implementierung gestützten plausiblen Einordnung und einer nicht nachgewiesenen kausalen Erklärung. Die konkrete Datenmodellierung, Indexierung, Transaktionslogik und Verbindungsverwaltung werden in den folgenden Abschnitten dokumentiert. Die Ergebnisse werden in Kapitel 3 anhand derselben Dimensionen eingeordnet.

2.3. Vergleichsdesign und Rahmenbedingungen

Auf Grundlage des in Abschnitt 2.1 hergeleiteten Lastprofils und der in Abschnitt 2.2 beschriebenen Bewertungslogik wird der empirische Vergleich als kontrollierte, datenbanknahe Lastuntersuchung aufgebaut. Ziel ist ein reproduzierbarer Vergleich von PostgreSQL und MongoDB unter demselben fachlichen E-Commerce-nahen Lastprofil. Die Untersuchung gliedert sich in vier getrennt ausgewertete Bereiche: Referenzreihe, replizierter Hochlastbereich, explorative Grenzlast und Datenmengen-Sensitivitätsanalyse.

Tabelle 2.4 fasst die Untersuchungsbereiche zusammen. Die Trennung ist methodisch wichtig, weil die Bereiche unterschiedliche Funktionen erfüllen und nicht mit derselben Aussagekraft interpretiert werden.

Tabelle 2.4.: Untersuchungsbereiche des empirischen Vergleichs

Untersuchungs-bereich	**Datengrundlage**	**Laststufen, Wiederholungen und Funktion**
Referenzreihe	`medium`	5, 10, 25, 50, 75 und 100 Locust-User; jeweils $n = 3$ je System. Kontrollierter Ausgangspunkt zur Einordnung niedrigerer und mittlerer Laststufen.
Replizierter Hochlastbereich	`medium`	500 und 1.000 Locust-User; jeweils $n = 3$ je System. Zentraler Hochlastvergleich mit wiederholten Messläufen und aggregierten Kennzahlen.
Explorative Grenzlast	`medium`	2.500 und 5.000 Locust-User; jeweils $n = 1$ je System. Ergänzende Betrachtung sehr hoher Laststufen als explorative Einzelmessungen ohne Streuungsaussage.
Datenmengen-Sensitivitätsanalyse	`medium` und `large_catalog`	500 Locust-User; jeweils $n = 3$ je System und Datensatzprofil. Separater Vergleich des Einflusses eines größeren Produktkatalogs bei konstanter Nutzer- und Warenkorbbasis.

Vor jedem Messlauf wird die Datenbank auf einen definierten Ausgangszustand zurückgesetzt und mit dem jeweiligen Datensatzprofil neu befüllt. Dadurch wirken sich Schreiboperationen aus vorherigen Messläufen, insbesondere Warenkorbänderungen und Checkout-Operationen, nicht auf nachfolgende Läufe aus. Während eines Messlaufs ist jeweils nur das untersuchte Datenbanksystem aktiv. Die Last wird durch Locust erzeugt; Nutzerzahl, Spawn-Rate, Laufzeit und Wiederholung werden je Versuchsreihe festgelegt.

Die standardisierte Versuchspipeline folgt in allen Messreihen demselben Grundablauf. Tabelle 2.5 fasst die zentralen Phasen zusammen. Die ausführliche Prozessdarstellung ist im Anhang A.1 dokumentiert.

Tabelle 2.5.: Kompakte Versuchspipeline der empirischen Untersuchung

Phase	Zentrale Schritte
Versuchsplanung	Untersuchungsbereich, Datenbanksystem, Datensatzprofil, Laststufe, Spawn-Rate, Laufzeit und Wiederholung festlegen.
Datengrundlage und Seeding	CSV-Datensatz auswählen, Datenbank zurücksetzen, PostgreSQL- beziehungsweise MongoDB-Seed-Skript ausführen und definierten Ausgangszustand herstellen.
Lasttestdurchführung	Genau ein Datenbanksystem starten, Locust im Headless-Modus ausführen und sechs E-Commerce-nahe Operationen gemäß Lastprofil messen.
Rohdatensicherung und Validierung	Locust-CSV-Dateien sichern, Zieluserphase prüfen, Failure- und Exception-Dateien kontrollieren sowie Sanity-Checks durchführen.
Aggregation und Auswertung	Replizierte Reihen aggregieren, explorative Grenzlast getrennt einordnen, Diagramme erzeugen und Ergebnisse anhand der definierten Metriken interpretieren.

2.3.1. Messaufbau und Testumgebung

Die empirischen Messungen wurden auf einer lokalen Einzelrechnerumgebung durchgeführt. Lastgenerator und jeweils untersuchtes Datenbanksystem liefen auf demselben physischen Rechner. Dadurch konnten die Messläufe kontrolliert und wiederholbar durchgeführt werden; zugleich sind die Ergebnisse an diese konkrete Hardware- und Softwareumgebung gebunden. Tabelle 2.6 fasst die wichtigsten Eigenschaften des Hostsystems zusammen.

Tabelle 2.6.: Lokale Hardware- und Softwareumgebung

Komponente	Ausprägung
Betriebssystem	Microsoft Windows 11 Home, Version 25H2, Betriebssystembuild 26200.8655
Prozessor	Intel Core i7-1165G7 der 11. Generation mit 2,80 GHz Basistakt
Physischer Arbeitsspeicher	16 GB
Docker	Docker 29.4.3, Build `055a478`
Python	Python 3.12.3
Locust	Locust 2.44.0

Die Datenbanksysteme wurden mit Docker Compose als getrennte Dienste bereitgestellt. Während eines Messlaufs war jeweils nur das untersuchte Datenbanksystem aktiv, sodass PostgreSQL und MongoDB nicht gleichzeitig um CPU-, Arbeitsspeicher- oder Massenspeicherressourcen konkurrierten. Beide Datenbankcontainer erhielten dieselben Ressourcenlimits. Tabelle 2.7 fasst die für die Messungen relevanten Parameter zusammen.

Tabelle 2.7.: Container- und Datenbankkonfiguration

Parameter	**PostgreSQL**	**MongoDB**
Container-Image	`postgres:18.4-bookworm`	`mongodb/mongodb-community-server:8.0.23-ubi8-slim`
CPU-Limit	4 CPUs	4 CPUs
RAM-Limit	6 GiB	6 GiB
Datenbankport	5432	27017
Speicherbezogene Konfiguration	`shared_buffers=1GB` `work_mem=16MB`	WiredTiger-Cache von 2 GB
Weitere relevante Einstellung	`synchronous_commit=on`	Single-Node-Replikatsatz `rs0`

MongoDB wurde als Single-Node-Replikatsatz betrieben, weil die checkoutnahe Operation eine Transaktion über mehrere Dokumente und Collections verwendet. Der Replikatsatz dient in diesem Aufbau nicht als verteilte Hochverfügbarkeitslösung, sondern stellt die für die verwendete Transaktionslogik erforderliche Betriebsform bereit.

Für die Lastgenerierung wurde Locust eingesetzt. Ausschlaggebend war, dass die fachlichen Operationen direkt in Python implementiert und über die Datenbanktreiber `psycopg` und `pymongo` ausgeführt werden konnten. Dadurch ließen sich dieselben E-Commerce-nahen Operationen für PostgreSQL und MongoDB programmierbar abbilden, einschließlich der systemspezifischen Checkout- und Transaktionslogik. Alternativen wie Apache JMeter oder Grafana k6 wären grundsätzlich für Lasttests geeignet, hätten für diesen direkten datenbanknahen Vergleich jedoch zusätzliche Skript-, Erweiterungs- oder Protokollunterstützung erfordert [17, 18, 19].

Im Unterschied zu einem typischen HTTP-Lasttest wurde kein `HttpUser` verwendet. Die simulierten Nutzer führten die Datenbankoperationen direkt über die jeweiligen Treiber aus. Die erfassten Antwortzeiten beschreiben daher datenbanknahe Operationen einschließlich clientseitiger Treiber- und Transaktionslogik, aber keine End-to-End-Antwortzeiten eines vollständigen Online-Shops. Die wissenschaftlich ausgewerteten Messläufe wurden im Headless-Modus ausgeführt und als Locust-CSV-Dateien exportiert [20].

Verbindungsverwaltung und Konfigurationswechsel.

Die Verbindungsverwaltung unterscheidet sich zwischen den Datenbanksystemen und teilweise zwischen den Versuchsreihen. Tabelle 2.8 stellt die final ausgewerteten Konfigurationen gegenüber.

Tabelle 2.8.: Verbindungsstrategien der ausgewerteten Versuchsreihen

Versuchsreihe	**PostgreSQL `max_connections`**	**PostgreSQL-Clientstrategie**	**MongoDB-Clientstrategie**
Referenzreihe	200	eine eigene Verbindung pro Locust-User	ein eigener `MongoClient` pro Locust-User
Replizierter Hochlastbereich	6.000	eine eigene Verbindung pro Locust-User	gemeinsamer `MongoClient` mit `maxPoolSize=1000`
Explorative Grenzlast	6.000	eine eigene Verbindung pro Locust-User	gemeinsamer `MongoClient` mit `maxPoolSize=1000`
Datenmengen-Sensitivitätsanalyse	6.000	eine eigene Verbindung pro Locust-User	gemeinsamer `MongoClient` mit `maxPoolSize=1000`

PostgreSQL öffnete beim Start jedes simulierten Nutzers eine eigene Datenbankverbindung und schloss sie beim Beenden dieses Nutzers. Für die Referenzreihe war die zulässige Verbindungszahl auf 200 begrenzt. Für die späteren Messreihen wurde `max_connections` auf 6.000 erhöht, damit die höheren Nutzerzahlen nicht bereits durch die ursprüngliche Verbindungsgrenze verhindert wurden. Die speicherbezogenen Parameter `shared_buffers` und `work_mem` blieben unverändert.

In der MongoDB-Referenzreihe wurde pro Locust-User ein eigener `MongoClient` erzeugt. Für die final ausgewertete Hochlast-, Grenzlast- und Sensitivitätsreihe wurde dagegen ein gemeinsam initialisierter `MongoClient` mit einem Connection Pool von maximal 1.000 Verbindungen verwendet. Dieser Wechsel war notwendig, um die höheren Laststufen technisch stabil ausführen zu können. Deshalb werden direkte Aussagen über den Übergang von 100 auf 500 User unter Berücksichtigung dieses Konfigurationswechsels formuliert.

2.3.2. Datenmodell, Datengrundlage und E-Commerce-Lastprofil

Für den empirischen Vergleich wird ein vereinfachtes E-Commerce-Modell verwendet. Es umfasst die für das Lastprofil relevanten Datenbereiche Kategorien, Produkte, Bestände, Kunden, Warenkörbe, Warenkorbpositionen, Bestellungen und Bestellpositionen. Beide Datenbanksysteme bearbeiten damit dieselben fachlichen Aufgaben, speichern die Daten jedoch entsprechend ihres jeweiligen Datenmodells.

PostgreSQL bildet die Daten relational mit Tabellen, Primärschlüsseln, Fremdschlüsseln, Constraints und Indexen ab. MongoDB verwendet Collections, in denen Bestandsinformationen in Produktdokumente sowie Warenkorb- und Bestellpositionen in die jeweiligen Warenkorb- beziehungsweise Bestelldokumente eingebettet werden. Die dokumentenorientierte Modellierung folgt dem Prinzip, regelmäßig gemeinsam verwendete Informationen möglichst gemeinsam

abzulegen [15]. Die konkrete technische Umsetzung beider Datenmodelle wird in Abschnitt 2.4 und Abschnitt 2.5 beschrieben.

Die Operationen sind fachlich vergleichbar, aber technisch systemspezifisch umgesetzt. Eine Produktdetailabfrage liefert beispielsweise in beiden Systemen Produkt-, Preis- und Bestandsinformationen. PostgreSQL verbindet dafür `products` und `inventory`, während MongoDB die entsprechenden Informationen aus einem Produktdokument liest. Die Indexierung unterstützt die für das Lastprofil relevanten Zugriffspfade und wird nicht als eigene Versuchsvariable variiert.

Synthetische Datengrundlage.

Die gemeinsame Datengrundlage wird mit einem Python-Skript synthetisch erzeugt. Ein fester Zufalls-Seed und ein fester initialer Zeitstempel sorgen dafür, dass PostgreSQL und MongoDB aus denselben fachlichen Ausgangsdaten befüllt werden. Die CSV-Dateien dienen dabei als datenbankunabhängige Zwischenform und werden anschließend durch separate Seed-Skripte in die relationale beziehungsweise dokumentenorientierte Struktur übertragen.

Für jedes Produkt werden unter anderem Produkt-ID, Artikelnummer, Kategorie, Marke, Preis und initialer Bestand erzeugt. Für jeden initialen Kunden wird ein aktiver Warenkorb angelegt; jeder Warenkorb enthält zufällig eine bis drei Positionen. Initiale Bestellungen werden nicht erzeugt, da Bestellungen erst durch die checkoutnahen Operationen während der Messläufe entstehen sollen. Vor jedem Messlauf wird die jeweilige Datenbank zurückgesetzt und erneut aus dem gewählten Datensatzprofil befüllt.

Datensatzprofile.

Für die ausgewerteten Messreihen werden die Profile `medium` und `large_catalog` verwendet. Tabelle 2.9 stellt die relevanten Größen gegenüber.

Tabelle 2.9.: Datensatzprofile der empirischen Untersuchung

Datenbereich	**`medium`**	**`large_catalog`**
Kategorien	25	25
Produkte	50.000	200.000
Bestandsdatensätze	50.000	200.000
Kunden	5.000	5.000
Aktive Warenkörbe	5.000	5.000
Initiale Warenkorbpositionen	1–3 je Warenkorb	1–3 je Warenkorb
Initiale Bestellungen	0	0

Das Profil `medium` bildet die Datengrundlage der Referenz-, Hochlast- und Grenzlastreihen. Es stellt einen nichttrivialen, aber lokal reproduzierbar ausführbaren Ausgangszustand bereit. Das Profil `large_catalog` wird ausschließlich für die Datenmengen-Sensitivitätsanalyse verwendet. Es erhöht die Zahl der Produkte und Bestandsdatensätze auf 200.000, während Kategorien, Kunden und Warenkörbe konstant bleiben. Dadurch wird gezielt betrachtet, wie die beiden Implementierungen bei konstanter Kunden- und Warenkorbbasis auf einen größeren Produktkatalog reagieren.

E-Commerce-nahes Lastprofil.

Das Lastprofil besteht aus sechs fachlichen Operationstypen. Sie decken zentrale datenbanknahe Nutzungsmuster eines Online-Shops ab: lesende Katalogzugriffe, punktuelle Produkt- und Bestandsabfragen, Warenkorbzugriffe, Warenkorbänderungen und einen vereinfachten checkoutnahen Transaktionsablauf. Tabelle 2.10 fasst Operationen und Gewichtungen zusammen.

Tabelle 2.10.: Operationen und Gewichtungen des E-Commerce-nahen Lastprofils

Operation	**Gewicht**	**Fachliche und datenbanknahe Bedeutung**
`product_list`	35 %	Produktliste anhand von Kategorie- und Preisfiltern; lesender Zugriff mit Filterung, Sortierung und Begrenzung der Ergebnismenge.
`product_detail`	25 %	Abruf eines einzelnen Produkts einschließlich Preis und Bestand; punktueller produktbezogener Lesezugriff.
`price_availability`	15 %	Prüfung von Preis und Verfügbarkeit eines Produkts; kurzer produktbezogener Lesezugriff.
`cart_read`	5 %	Lesen eines aktiven Warenkorbs mit Positionen; Zugriff auf Warenkorb- und Positionsdaten.
`cart_update`	15 %	Hinzufügen eines Produkts oder Erhöhen einer vorhandenen Menge; schreibender beziehungsweise aktualisierender Warenkorbzugriff.
`checkout`	5 %	Vereinfachter Abschluss eines aktiven Warenkorbs; transaktionsnaher Ablauf mit mehreren Lese- und Schreibschritten.

Die Gewichtung ist eine kontrollierte Modellierungsannahme und nicht aus Produktivprotokollen eines konkreten Online-Shops kalibriert. Mit 80 % lesenden und 20 % schreibenden beziehungsweise transaktionsnahen Operationen ist das Profil überwiegend leselastig. Produktlisten und Produktdetails erhalten den größten Anteil, weil das Suchen und Betrachten von Produkten im modellierten Kaufprozess häufiger auftreten soll als Warenkorb- und Checkout-Vorgänge. Gleichzeitig sorgen Warenkorbänderungen und Checkout-Operationen dafür, dass regelmäßig konkurrierende Schreib- und Transaktionssituationen entstehen.

Nach jeder Operation wartet ein simulierter Nutzer zufällig zwischen 0,1 und 0,5 Sekunden. Die Gewichtungen legen keine feste Requestzahl fest, sondern die relative Wahrscheinlichkeit, mit der Locust die Operationstypen auswählt. Innerhalb der Operationen werden Produkte, Kunden oder Warenkörbe zufallsbasiert ausgewählt, damit sich die Last nicht dauerhaft auf wenige feste Datensätze konzentriert.

Unter `checkout` wird ein vereinfachter Bestellabschluss verstanden: Ein aktiver Warenkorb mit Positionen wird ausgewählt, Bestände werden geprüft, eine Bestellung wird angelegt, betroffene Bestände werden reduziert und der bisherige Warenkorb wird abgeschlossen. Anschließend wird für denselben Kunden ein neuer aktiver Warenkorb erzeugt. Zahlungsabwicklung, Versand, Gutscheine, Steuern und externe Dienste sind nicht Bestandteil dieses Modells; die Operation fokussiert die datenbankrelevanten Schritte des Bestellabschlusses.

2.3.3. Metriken, Aggregation und Bewertungskriterien

Die Bewertung der Datenbanksysteme erfolgt anhand der in Abschnitt 2.2.1 hergeleiteten mehrdimensionalen Bewertungslogik. Als zentrale Messgrößen werden Durchsatz, Median-Latenz, p95-Latenz, p99-Latenz und Fehlerrate verwendet. Keine dieser Kennzahlen wird isoliert als Nachweis eines günstigen oder ungünstigen Systemverhaltens interpretiert.

Die Locust-Skripte messen die Laufzeit jeder fachlichen Operation unmittelbar vor dem Aufruf der datenbanknahen Logik bis zu deren erfolgreichem Abschluss oder endgültigem Fehler. Der Messbereich umfasst damit die Datenbankzugriffe, die Treiberverarbeitung und die innerhalb der Operation ausgeführte Transaktionslogik. Nicht enthalten sind Anwendungsschicht, HTTP-Kommunikation, Frontend, externe Dienste sowie die zwischen zwei Tasks vorgesehene Locust-Wartezeit von 0,1 bis 0,5 Sekunden. Die Messwerte beschreiben daher datenbanknahe Operationen aus Sicht des Lastgenerators und keine End-to-End-Antwortzeiten eines vollständigen Online-Shops.

Der Durchsatz entspricht der Locust-Kennzahl `Requests/s` und beschreibt die pro Sekunde erfassten abgeschlossenen Request-Ereignisse. Er ist von der eingestellten Nutzerzahl zu unterscheiden: Die Nutzerzahl bestimmt die angebotene Parallelität, während der tatsächlich erreichte Durchsatz von Antwortzeiten, Task-Gewichtung, Wartezeiten und Datenbankverhalten abhängt. Median, p95 und p99 beschreiben die Latenzverteilung eines Messlaufs. Der Median steht für das typische Antwortzeitverhalten, während p95 und p99 langsame Operationen im oberen Bereich der Verteilung sichtbar machen [14]. Die Locust-Statistik wurde pro Lauf sowohl für einzelne Operationstypen als auch aggregiert über das gesamte Lastprofil im CSV-Format gespeichert [20].

Die Fehlerrate beschreibt den Anteil endgültig fehlgeschlagener Operationen an allen von Locust erfassten Operationen. Bei MongoDB werden transiente Checkout-Konflikte zunächst innerhalb der Operation erneut ausgeführt. Gelingt ein Wiederholungsversuch, wird die Operation als erfolgreich gezählt; die zusätzliche Ausführungszeit bleibt jedoch in der Latenz enthalten. Erst wenn auch der letzte vorgesehene Versuch scheitert, wird ein Failure registriert.

Von Failures zu unterscheiden sind Einträge in der Locust-Exception-Datei, die auf unbehandelte Fehler in der Task-Ausführung oder Lasttestinstrumentierung hinweisen können. Die final ausgewerteten Läufe werden deshalb anhand der Failure- und Exception-Dateien geprüft.

Für die Referenzreihe, den replizierten Hochlastbereich und die Datenmengen-Sensitivitätsanalyse liegen pro System und Konfiguration jeweils drei Messläufe vor. Für diese replizierten Reihen werden die aggregierten Locust-Werte jedes einzelnen Laufs übernommen und anschließend Mittelwert, Standardabweichung, Minimum und Maximum über die Wiederholungen berechnet. Median, p95 und p99 werden dabei als Mittelwerte der von Locust je Lauf berechneten Perzentile ausgewiesen. Es handelt sich somit nicht um neu berechnete Perzentile einer gepoolten Gesamtheit.

Für die explorative Grenzlast liegt je System und Laststufe ein Messlauf vor. Diese Messpunkte werden als Einzelwerte ohne Streuungsaussage behandelt. Tabelle 2.11 dokumentiert die verwendeten Auswertungsfenster der einzelnen Messreihen.

Tabelle 2.11.: Auswertungsfenster der untersuchten Messreihen

Versuchsbereich	*n*	**Auswertungsgrundlage**
Referenzreihe, 5 bis 100 User	3	Aggregierte Locust-Statistik des vollständigen zehnminütigen Messlaufs.
Replizierter Hochlastbereich, 500 und 1.000 User	3	Aggregierte Locust-Statistik des vollständigen zehnminütigen Messlaufs.
PostgreSQL-Grenzlast, 2.500 und 5.000 User	1	Aggregierte Locust-Statistik des vollständigen zehnminütigen Messlaufs.
MongoDB-Grenzlast, 2.500 und 5.000 User	1	Stabile Phase, in der die jeweilige Zieluserzahl erreicht und gehalten wurde.
Datenmengen-Sensitivitätsanalyse, 500 User	3	Aggregierte Locust-Statistik des vollständigen zehnminütigen Messlaufs für die Datensatzprofile `medium` und `large_catalog`.

Bei den MongoDB-Grenzlastläufen blieben nach Ablauf der vorgesehenen zehn Minuten noch lang laufende Operationen offen. Für Durchsatz und Fehlerrate der MongoDB-Grenzlast werden deshalb die Differenzen der kumulierten Request- und Failure-Zähler innerhalb der stabilen Zieluserphase verwendet. Die Latenzperzentile werden dem letzten Locust-Verlaufspunkt entnommen, an dem die Zieluserzahl noch gehalten wurde. Diese Werte sind kumulierte Locust-Perzentile bis zu diesem Zeitpunkt und keine ausschließlich aus den Einzelanfragen der stabilen Phase neu berechneten Perzentile. Die PostgreSQL-Grenzlastwerte beruhen dagegen auf der finalen Statistik des vollständigen Laufs. Die unterschiedliche Auswertungsgrundlage wird in den Ergebnistabellen und Abbildungsunterschriften ausgewiesen.

Für die Datenmengen-Sensitivitätsanalyse werden die aggregierten Mittelwerte des Profils `large_catalog` mit den entsprechenden `medium`-Werten bei 500 Usern verglichen. Die relative Veränderung wird als prozentuale Änderung gegenüber dem `medium`-Profil berechnet. Ein negativer Wert beschreibt bei der Durchsatzmetrik einen Rückgang, während ein positiver Wert bei den Latenzmetriken eine Zunahme der Antwortzeit bezeichnet.

Stabilität wird anhand des gemeinsamen Verlaufs von Durchsatz, Median-, p95- und p99-Latenz sowie Fehlerrate eingeordnet. Ein Sättigungs- beziehungsweise Degradationsbereich wird angenommen, wenn eine höhere Nutzerzahl keinen entsprechenden Durchsatzgewinn mehr erzeugt oder mit einem Durchsatzrückgang einhergeht, während Median- oder Tail-Latenzen deutlich ansteigen. Endgültige Failures sind ein zusätzlicher, aber nicht der einzige Hinweis auf Überlastung; eine Fehlerrate von 0 % wird daher nicht automatisch als hinreichende Systemleistung interpretiert.

Die Bewertung erfolgt vergleichend innerhalb des dokumentierten Versuchsaufbaus. Gemessene Ergebnisse, plausible Erklärungen anhand von Datenmodell, Indexierung, Transaktionsverarbeitung oder Verbindungsverwaltung und nicht nachgewiesene Ursachen werden sprachlich voneinander getrennt. Ergänzend beobachtete Ressourcenwerte wie CPU, Arbeitsspeicher, Prozessanzahl und I/O dienen ausschließlich der Plausibilisierung, da sie nicht als kontinuierliche, standardisierte Zeitreihe erhoben wurden.

2.3.4. Geltungsbereich und Interpretationsannahmen

Die vorangegangenen Abschnitte legen den Geltungsbereich der empirischen Untersuchung fest. Der Vergleich bezieht sich auf datenbanknahe E-Commerce-Operationen, die direkt über die jeweiligen Datenbanktreiber ausgeführt werden. Die Messwerte beschreiben damit das Verhalten der konkret realisierten Datenbankzugriffe aus Sicht des Lastgenerators. Performancewerte sind dabei grundsätzlich an Workload, Datenmodellierung, Systemkonfiguration und Testumgebung gebunden [9].

PostgreSQL und MongoDB bearbeiten in den Messläufen dieselben fachlichen Aufgaben, etwa Produktabfragen, Warenkorboperationen und Checkout-Abläufe. Die technische Umsetzung unterscheidet sich jedoch aufgrund der relationalen beziehungsweise dokumentenorientierten Modellierung. Vergleichbar ist daher der fachliche Zweck der Operationen; die gemessenen Unterschiede beziehen sich auf die jeweils dokumentierte Implementierung mit ihren Datenmodellen, Indexen, Abfragen, Transaktionsmechanismen und Clientstrategien.

Die Ergebnisse werden getrennt nach Referenzreihe, repliziertem Hochlastbereich, explorativer Grenzlast und Datenmengen-Sensitivitätsanalyse interpretiert. Replizierte Reihen mit $n = 3$ ermöglichen eine Auswertung über Mittelwerte und Streuungsmaße, während die Grenzlastläufe mit $n = 1$ als explorative Einzelmessungen ohne Streuungsaussage behandelt werden. Die späteren Ergebnisinterpretationen beziehen sich damit auf den dokumentierten lokalen Einzelknotenaufbau, die gewählten Datensatzprofile, das definierte Lastprofil, die verwendeten Datenmodelle und die festgelegten Systemkonfigurationen. Gemessene Ergebnisse, plausible technische Interpretationen und nicht nachgewiesene Ursachen werden in der Auswertung sprachlich voneinander getrennt.

2.4. PostgreSQL als relationales Vergleichssystem

PostgreSQL wird in dieser Untersuchung nicht als allgemeiner Vertreter aller relationalen Datenbanksysteme bewertet, sondern als konkret konfigurierte und implementierte Vergleichsinstanz. Maßgeblich sind das verwendete relationale Schema, die eingerichteten Indexe, die SQL-Abfragen, die Transaktionsbehandlung und die in Abschnitt 2.3.1 dokumentierte Verbindungsstrategie.

Relationale Abbildung und Integritätsregeln.

Das fachliche E-Commerce-Modell wird in PostgreSQL durch die acht Tabellen `categories`, `products`, `inventory`, `customers`, `carts`, `cart_items`, `orders` und `order_items` abgebildet. Kategorien, Produkte, Kunden, Warenkörbe und Bestellungen besitzen eigene Primärschlüssel. Beziehungen zwischen den Entitäten werden über Fremdschlüssel hergestellt.

Produkt- und Bestandsinformationen werden in den getrennten Tabellen `products` und `inventory` gespeichert. Warenkorb- und Bestellpositionen liegen ebenfalls in separaten Tabellen. Für Produktdetail-, Warenkorb- und Checkout-Operationen sind daher teilweise Joins zwischen mehreren Tabellen erforderlich.

Das Schema enthält Integritätsbedingungen für nichtnegative Preise und Bestellbeträge, nichtnegative Bestände sowie positive Positionsmengen. Die Kombination aus Warenkorb- und Produkt-ID bildet den Primärschlüssel von `cart_items`. Dadurch kann ein Produkt pro Warenkorb höchstens einmal als eigene Position gespeichert werden. Eine erneute Warenkorbänderung erhöht die vorhandene Menge, anstatt eine weitere Position für dasselbe Produkt anzulegen.

Die relationale Datenbank übernimmt damit einen Teil der Konsistenzsicherung unmittelbar durch Schlüssel- und `CHECK`-Constraints. Diese Regeln sind Bestandteil der gemessenen Implementierung.

Indexierung.

Die eingerichteten Indexe orientieren sich an den Zugriffsmustern des Lastprofils. Für Produktlisten existiert ein zusammengesetzter Index auf Kategorie und Preis. Ein weiterer Index unterstützt Abfragen nach aktiven Produkten und Kategorie. Aktive Warenkörbe können über Kunde und Status gesucht werden. Für Warenkorbpositionen, Bestellungen und Bestellpositionen wurden zusätzliche Zugriffspfade auf Produkt- beziehungsweise Kundenkennungen eingerichtet.

Primärschlüssel und Eindeutigkeitsbedingungen erzeugen in PostgreSQL weitere Indexstrukturen automatisch. Die Indexierung kann Lesezugriffe beschleunigen, verursacht bei schreibenden Operationen jedoch zugleich Aktualisierungsaufwand [21]. Die Untersuchung variiert die Indexkonfiguration nicht. Die gemessenen Ergebnisse gelten daher für das festgelegte Schema einschließlich der darin enthaltenen Indexe.

Treiber- und Verbindungsmodell.

Die Lastgenerierung greift über den Python-Treiber `psycopg` direkt auf PostgreSQL zu. Beim Start eines Locust-Users wird eine eigene Datenbankverbindung geöffnet und beim Beenden des Users wieder geschlossen. Der Verbindungsaufbau liegt außerhalb der einzelnen Request-Messungen. Die gemessenen Antwortzeiten enthalten somit die über eine bereits bestehende Verbindung ausgeführten SQL-Anweisungen und die zugehörige Treiberverarbeitung.

Die Verbindung wird mit aktiviertem Autocommit betrieben. Einzelne Leseoperationen und die Warenkorbänderung werden daher nicht innerhalb einer über die gesamte Task-Ausführung gespannten expliziten Transaktion zusammengefasst. Bei `cart_update` werden das Einfügen beziehungsweise Aktualisieren der Warenkorbposition und die anschließende Aktualisierung des Warenkorbs als aufeinanderfolgende SQL-Anweisungen ausgeführt.

Der Checkout bildet eine Ausnahme. Er wird über einen expliziten Transaktionskontext ausgeführt, sodass seine zusammengehörigen Lese- und Schreiboperationen gemeinsam bestätigt oder bei einem Fehler vollständig zurückgerollt werden.

Umsetzung der Lastprofiloperationen.

Die sechs in Tabelle 2.10 definierten Lastprofiloperationen werden in PostgreSQL mit SQL-Abfragen auf dem relationalen Schema umgesetzt. Die produktbezogenen Operationen `product_list`, `product_detail` und `price_availability` greifen auf die Tabellen `products` und `inventory` zu. Die Produktliste filtert aktive Produkte nach Kategorie und Preisbereich, sortiert nach Preis und begrenzt die Ergebnismenge auf 20 Zeilen. Produktdetail und Preis-/Bestandsabfrage verwenden punktuelle Zugriffe über zufällig ausgewählte Produkt-IDs.

Die Warenkorboperation `cart_read` sucht einen aktiven Warenkorb nach Kunde und Status und liest die zugehörigen Positionen über `cart_items`. Bei `cart_update` wird zunächst ein zufälliger aktiver Warenkorb ausgewählt. Anschließend wird ein zufällig bestimmtes Produkt eingefügt oder die vorhandene Positionsmenge über `INSERT ... ON CONFLICT DO UPDATE` erhöht. Die zufällige Auswahl ist Teil der konkreten Implementierung; ihre möglichen Kosten werden nicht isoliert von den übrigen Operationsbestandteilen gemessen.

Checkout-Transaktion.

Die PostgreSQL-Checkout-Operation führt mehrere zusammengehörige Änderungen innerhalb einer Transaktion aus. Dabei wird ein aktiver Warenkorb mit mindestens einer Position ausgewählt, der Warenkorb wird gesperrt, die zugehörigen Bestandszeilen werden gelesen und ebenfalls gesperrt, die verfügbaren Bestandsmengen werden geprüft, eine Bestellung mit Bestellpositionen wird angelegt, die Bestände werden reduziert und der bisherige Warenkorb wird als `checked_out` markiert. Anschließend wird für denselben Kunden ein neuer aktiver Warenkorb angelegt.

Für die konkurrierende Verarbeitung mehrerer Checkout-Operationen werden Zeilensperren verwendet. Mit `FOR UPDATE` werden ausgewählte Zeilen bis zum Ende der Transaktion für konkurrierende Änderungen gesperrt. Der Zusatz `SKIP LOCKED` bewirkt, dass bereits durch eine andere Transaktion gesperrte Warenkörbe bei der Auswahl übersprungen werden [22]. Dadurch können parallele Checkout-Operationen unterschiedliche verfügbare Warenkörbe bearbeiten, anstatt auf dieselbe Sperre zu warten.

Die Bestandsreduzierung erfolgt zusätzlich mit einer Bedingung auf einen weiterhin ausreichenden Bestand. Schlägt eine Aktualisierung fehl oder tritt während des Ablaufs eine Exception auf, wird die gesamte Transaktion zurückgerollt. Dadurch entsteht weder eine unvollständige Bestellung noch eine nur teilweise durchgeführte Bestandsänderung. Das anschließende Anlegen eines neuen aktiven Warenkorbs hält die Zahl aktiver Warenkörbe grundsätzlich konstant, sodass auch in längeren Messläufen weiterhin Warenkorb- und Checkout-Operationen ausgeführt werden können.

Nebenläufigkeit und methodische Einordnung.

PostgreSQL verwendet Multi-Version Concurrency Control, wodurch Transaktionen konsistente Datenstände sehen und viele Lese- und Schreiboperationen parallel ausgeführt werden können [16]. Die expliziten Sperren des Checkouts ergänzen dieses Verfahren dort, wo mehrere Transaktionen dieselben Warenkorb- oder Bestandszeilen verändern könnten.

Aus den verwendeten Mechanismen darf jedoch nicht unmittelbar auf die Ursache eines später gemessenen Durchsatz- oder Latenzverlaufs geschlossen werden. Neben MVCC und Sperren beeinflussen unter anderem die konkreten SQL-Abfragen, die zufällige Warenkorbauswahl, die Join-Operationen, die Indexierung, die Anzahl paralleler Verbindungen und die lokale Ressourcenumgebung das beobachtete Verhalten.

In der Referenzreihe war die zulässige Verbindungszahl auf 200 begrenzt. Für die Hochlast-, Grenzlast- und Sensitivitätsmessungen wurde `max_connections` auf 6.000 erhöht, während jeder Locust-User weiterhin eine eigene Verbindung verwendete. Ein zusätzlicher anwendungsseitiger Connection Pool wurde für PostgreSQL nicht eingesetzt. Diese Entscheidung ist Teil des untersuchten Aufbaus und begrenzt die Ergebnisse auf eine direkte Verbindungsstrategie.

Weitere PostgreSQL-Optimierungen, etwa ein Connection Pool, abweichende Speicherparameter, alternative SQL-Abfragen oder eine gezielte Indexoptimierung, werden nicht als Versuchsvariablen untersucht. Die Messergebnisse charakterisieren daher die hier dokumentierte relationale Umsetzung und nicht das maximal erreichbare Leistungsniveau von PostgreSQL.

2.5. MongoDB als dokumentenorientiertes Vergleichssystem

MongoDB wird in dieser Untersuchung nicht als allgemeiner Vertreter aller dokumentenorientierten Datenbanksysteme bewertet, sondern als konkret modellierte und konfigurierte Vergleichsinstanz. Maßgeblich sind die verwendete Dokumentstruktur, die eingerichteten Indexe, die MongoDB-Abfragen, die Transaktions- und Konfliktbehandlung sowie die in Abschnitt 2.3.1 dokumentierten Clientstrategien.

Dokumentenorientierte Abbildung.

Das fachliche E-Commerce-Modell wird in MongoDB durch die fünf Collections `categories`, `products`, `customers`, `carts` und `orders` abgebildet. Bestandsinformationen werden als Unterdokument `inventory` im jeweiligen Produktdokument gespeichert. Warenkorbpositionen und Bestellpositionen werden als `items`-Arrays in die zugehörigen Warenkorb- beziehungsweise Bestelldokumente eingebettet. Diese Modellierung folgt dem Grundsatz, Informationen, die regelmäßig gemeinsam gelesen oder verändert werden, innerhalb desselben Dokuments abzulegen [15, 23].

Zwischen den Collections bleiben weiterhin Referenzen bestehen. Ein Warenkorb enthält beispielsweise die Kennung seines Kunden, und die eingebetteten Positionen enthalten Produktkennungen. Diese Kennungen werden durch die Lasttestlogik aufgelöst. Die Konsistenz der erzeugten Daten und der während der Messläufe veränderten Bestände wird durch Seed-Logik, anwendungsseitige Prüfungen im Lasttest und ergänzende Sanity-Checks nach den Versuchsreihen abgesichert.

Indexierung.

Die MongoDB-Indexierung orientiert sich an den Zugriffsmustern des Lastprofils. Für Artikelnummern und Kunden-E-Mail-Adressen bestehen eindeutige Indexe. Produktlisten werden durch einen zusammengesetzten Index auf Kategorie und Preis sowie einen weiteren Index auf Aktivstatus und Kategorie unterstützt. Aktive Warenkörbe können über die Kombination aus Kundenkennung und Status gesucht werden.

Zusätzlich bestehen Multikey-Indexe auf den Produktkennungen innerhalb der eingebetteten Arrays von Warenkörben und Bestellungen. Für Bestellungen wurde außerdem ein zusammengesetzter Index auf Kundenkennung und Erstellungszeit angelegt. MongoDB-Indexe können Lesezugriffe beschleunigen, müssen bei Schreiboperationen jedoch ebenfalls aktualisiert werden [24, 25]. Die Indexkonfiguration wird im Versuch nicht variiert; die Ergebnisse gelten daher für die dokumentierte Collection- und Indexstruktur.

Treiber- und Clientmodell.

Der direkte Datenbankzugriff erfolgt über den Python-Treiber `pymongo`. Die Verbindungsstrategie unterscheidet sich zwischen der Referenzreihe und den späteren Versuchsreihen. In der Referenzreihe wurde für jeden Locust-User ein eigener `MongoClient` erzeugt. Für den replizierten Hochlastbereich, die explorative Grenzlast und die Datenmengen-Sensitivitätsanalyse wird dagegen ein gemeinsamer `MongoClient` verwendet, der einmalig beim Laden des Lasttestskripts initialisiert und von allen Locust-Usern gemeinsam genutzt wird.

Der gemeinsame Client ist mit `maxPoolSize=1000` konfiguriert. Dieser Wert beschreibt die Obergrenze des zugehörigen Verbindungspools und bedeutet nicht, dass von Beginn an 1.000 Verbindungen geöffnet werden. Der Aufbau des gemeinsamen Clients liegt außerhalb der Messzeit einzelner Operationen. Die geänderte Clientstrategie ermöglicht die höheren Laststufen mit stärkerer Wiederverwendung von Verbindungen, stellt aber zugleich einen methodischen Unterschied zwischen Referenz- und Hochlastreihe dar.

Umsetzung der Lastprofiloperationen.

Die sechs in Tabelle 2.10 definierten Lastprofiloperationen werden in MongoDB über Abfragen und Aktualisierungen auf den beschriebenen Collections umgesetzt. Die produktbezogenen Operationen `product_list`, `product_detail` und `price_availability` greifen jeweils auf die Collection `products` zu. Da der Bestand in das Produktdokument eingebettet ist, können Produkt- und Bestandsinformationen gemeinsam aus einem Dokument projiziert werden. Daraus folgt jedoch nicht automatisch ein schnelleres Verhalten gegenüber der relationalen Umsetzung, da auch Filterung, Indexauswahl, Dokumentgröße, Parallelität und weitere Faktoren die Laufzeit beeinflussen.

Bei `cart_read` wird ein aktiver Warenkorb über Kundenkennung und Status gesucht und zusammen mit den eingebetteten Positionen gelesen. `cart_update` sucht zunächst einen aktiven Warenkorb und ein Produkt. Anschließend wird entweder eine vorhandene Arrayposition über den positionalen Arrayoperator aktualisiert oder eine neue Position in das `items`-Array eingefügt. Im Unterschied zum zusammengesetzten Primärschlüssel der PostgreSQL-Tabelle `cart_items` besteht innerhalb des MongoDB-Arrays keine datenbankseitig konfigurierte Eindeutigkeitsbedingung für die Produktkennung. Die Vermeidung mehrfacher Positionen für dasselbe Produkt beruht daher auf der beschriebenen Anwendungslogik.

Checkout-Transaktion.

Der MongoDB-Checkout verändert mehrere Collections und Dokumente. Er wird deshalb innerhalb einer Session als Mehrdokumenttransaktion ausgeführt. Die verwendete Single-Node-Replikatsatzkonfiguration dient in diesem Aufbau der Unterstützung dieser Transaktionslogik und wird nicht als Untersuchung von Replikation oder Hochverfügbarkeit interpretiert.

Der Checkout wählt einen aktiven Warenkorb mit mindestens einer Position aus, setzt dessen Status innerhalb der Transaktion bedingt von `active` auf `checking_out`, prüft die betroffenen Produktbestände, fügt ein Bestelldokument mit eingebetteten Positionen ein, reduziert die Bestände bedingt und markiert den bisherigen Warenkorb als `checked_out`. Anschließend wird für denselben Kunden ein neuer aktiver Warenkorb angelegt. Dadurch bleiben auch in längeren Messläufen weiterhin aktive Warenkörbe für nachfolgende Warenkorb- und Checkout-Operationen verfügbar.

Die bedingte Statusänderung stellt sicher, dass der Warenkorb zum Zeitpunkt der Aktualisierung weiterhin aktiv ist. Auch die Bestandsreduzierung erfolgt nur, wenn der vorhandene Bestand mindestens der benötigten Menge entspricht. Wird ein Produkt nicht gefunden oder reicht der Bestand nicht aus, wird die Operation abgebrochen. Durch das Zurückrollen der Transaktion werden bereits ausgeführte Änderungen nicht dauerhaft übernommen.

Transiente Konflikte und Retry-Logik.

Bei konkurrierenden MongoDB-Transaktionen können Schreibkonflikte auftreten. MongoDB kennzeichnet bestimmte Fehler mit dem Label `TransientTransactionError`. Dieses Label zeigt an, dass eine erneute Ausführung der gesamten Transaktion erfolgreich sein kann [26]. Das Lasttestskript führt den Checkout deshalb bei entsprechend gekennzeichneten `PyMongoError`-Ausnahmen begrenzt erneut aus. Insgesamt sind maximal drei Versuche vorgesehen. Zwischen den Wiederholungen wird zunächst 0,05 Sekunden und vor einem dritten Versuch 0,10 Sekunden gewartet.

Ein Konflikt, der innerhalb der vorgesehenen Versuche erfolgreich behandelt wird, erscheint nicht als endgültiger Failure. Die Wiederholung und die Wartezeit befinden sich jedoch innerhalb der in Abschnitt 2.3.3 definierten Request-Messgrenze und erhöhen entsprechend die gemessene Latenz. Erst wenn auch der letzte Versuch scheitert, wird die Operation als Failure an Locust gemeldet. Die in den Hoch- und Grenzlastmessungen beobachteten endgültigen MongoDB-Checkout-Fehler sind daher nicht mit jedem intern aufgetretenen Schreibkonflikt gleichzusetzen.

Methodische Einordnung.

Die dokumentenorientierte Speicherung ermöglicht es, Produkt und Bestand sowie Warenkorb beziehungsweise Bestellung und ihre Positionen jeweils gemeinsam zu lesen. Gleichzeitig können Änderungen an eingebetteten Arrays und Mehrdokumenttransaktionen andere Zugriffsmuster und Konfliktsituationen erzeugen als die relationale PostgreSQL-Umsetzung.

Aus diesen Architekturunterschieden darf jedoch nicht unmittelbar auf die Ursache eines gemessenen Durchsatz-, Latenz- oder Fehlerverlaufs geschlossen werden. Das beobachtete Verhalten wird zusätzlich durch die konkreten Abfragen, die Dokumentgrößen, die Indexierung, die zufällige Datenauswahl, die Verbindungsverwaltung und die lokale Ressourcenumgebung beeinflusst.

Die finalen MongoDB-Hochlast-, Grenzlast- und Sensitivitätsläufe verwenden den gemeinsamen Client mit Connection Pool. Frühere Hochlastvorläufe mit einem eigenen Client pro Locust-User werden nicht in die finale Hochlastauswertung einbezogen. Die Referenzreihe verwendet dagegen die ursprüngliche Clientstrategie und wird als eigenständiger Untersuchungsbereich behandelt.

Weitere Varianten wie alternative Dokumentstrukturen, andere Poolobergrenzen, eine veränderte Retry-Strategie, zusätzliche Schema-Validierung, Sharding oder ein Mehrknoten-Replikatsatz werden nicht als Versuchsvariablen untersucht. Die Messergebnisse charakterisieren daher die hier dokumentierte MongoDB-Implementierung und nicht das maximal erreichbare Leistungsniveau des Datenbanksystems.

3. Empirische Evaluation

Aufbauend auf dem in Kapitel 2 beschriebenen konzeptionellen und methodischen Rahmen werden in diesem Kapitel die empirischen Messergebnisse dargestellt, gegenübergestellt und eingeordnet. Im Mittelpunkt steht die Frage, wie sich PostgreSQL und MongoDB unter einem kontrollierten E-Commerce-nahen Lastprofil bei steigender Parallelität und bei vergrößertem Produktkatalog verhalten.

Die Evaluation ist in vier Auswertungsbereiche gegliedert. Zunächst wird die Referenzreihe bei 5 bis 100 Locust-Usern betrachtet. Diese Reihe bildet den methodisch vollständig replizierten Ausgangspunkt des Vergleichs. Anschließend folgt der replizierte Hochlastbereich bei 500 und 1.000 Usern, der den Schwerpunkt der Lastbetrachtung darstellt. Ergänzend werden die explorativen Grenzlastläufe bei 2.500 und 5.000 Usern ausgewertet. Abschließend wird anhand einer Datenmengen-Sensitivitätsanalyse untersucht, wie sich eine Vergrößerung des Produktkatalogs bei konstanter Nutzerzahl auf die gemessenen Kennzahlen auswirkt.

Die Interpretation erfolgt jeweils direkt im Anschluss an die zugehörigen Messergebnisse. Dabei wird zwischen gemessenen Befunden, plausiblen Interpretationen und nicht nachgewiesenen Ursachen unterschieden. Die Reichweite der Ergebnisse und die methodischen Grenzen des Versuchsaufbaus werden anschließend in Kapitel 4 zusammengeführt.

3.1. Durchführung und Validierung der Messreihen

Für die empirische Evaluation wurde der in Abschnitt 2.3 beschriebene Messaufbau in konkrete Messreihen überführt. Die fachlichen Operationen, das Lastprofil, die Datensatzprofile, die Metriken und die Auswertungslogik wurden in Kapitel 2 definiert. Dieses Kapitel greift diese Grundlagen nur insoweit auf, wie es für die Einordnung der Ergebnisse notwendig ist.

Tabelle 3.1 fasst die ausgewerteten Messreihen zusammen. Alle Messläufe wurden mit einer vorgesehenen Laufzeit von zehn Minuten durchgeführt. Die replizierten Reihen werden über drei Wiederholungen je System und Konfiguration aggregiert; die Grenzlastläufe werden als explorative Einzelmessungen ohne Streuungsaussage interpretiert.

Vor jedem Messlauf wurde das jeweils getestete Datenbanksystem zurückgesetzt und aus der gemeinsamen CSV-Datengrundlage neu befüllt. Dadurch wurden Folgeeffekte vorheriger Schreib- und Checkout-Operationen vermieden. PostgreSQL und MongoDB wurden getrennt voneinander getestet; während eines Messlaufs war jeweils nur das untersuchte Datenbanksystem aktiv.

Tabelle 3.1.: Ausgewertete Messreihen der empirischen Evaluation

Auswertungs-bereich	**Datensatzprofil**	**Laststufen und Wiederholungen**	**Funktion in der Evaluation**
Referenzreihe	`medium`	5, 10, 25, 50, 75 und 100 Locust-User; $n = 3$	Replizierter Ausgangsbereich zur Einordnung von Durchsatz, Latenzen und Stabilität bei niedriger bis moderater Parallelität.
Replizierter Hochlastbereich	`medium`	500 und 1.000 Locust-User; $n = 3$	Zentraler Hochlastbereich der Arbeit mit aggregierten Kennzahlen über drei Wiederholungen je System und Laststufe.
Explorative Grenzlast	`medium`	2.500 und 5.000 Locust-User; $n = 1$	Ergänzende Untersuchung stark belasteter Grenzbereiche als Einzelmessungen ohne Streuungsaussage.
Datenmengen-Sensitivitätsanalyse	`medium` und `large_catalog`	500 Locust-User; $n = 3$ je Profil	Vergleich bei konstanter Laststufe zur Einordnung des Einflusses eines vergrößerten Produktkatalogs.

Die von Locust erzeugten CSV-Dateien wurden gesichert und mit den Analyse-Skripten zu aggregierten Ergebnisdateien verarbeitet. Für replizierte Messreihen wurden Mittelwerte, Standardabweichungen, Minimum und Maximum berechnet. Bei der explorativen Grenzlast werden Einzelwerte berichtet, da dort je System und Laststufe nur ein gültiger Lauf vorliegt.

Die technische Validierung der Läufe umfasste die Prüfung der erreichten Zieluserzahl, die Kontrolle der Failure- und Exception-Dateien sowie Sanity-Checks auf fachliche Plausibilität. Dabei wurden insbesondere Produkt- und Bestandszahlen, nichtnegative Lagerbestände sowie plausible Zahlen aktiver Warenkörbe, Bestellungen und Bestellpositionen geprüft. Bei den MongoDB-Grenzlastläufen bei 2.500 und 5.000 Usern wird die stabile Zieluserphase als Auswertungsfenster verwendet, weil nach Ablauf der zehnminütigen Zielphase noch lange offene Requests zurückliefen. Die übrigen Messreihen beruhen auf der vollständigen Locust-Statistik des jeweiligen Messlaufs.

Die vollständige Versuchspipeline und die ergänzenden digitalen Materialien sind in Anhang A.1 beziehungsweise Anhang A.2 dokumentiert. Die folgenden Abschnitte stellen die Ergebnisse der einzelnen Auswertungsbereiche dar und ordnen sie im Hinblick auf Durchsatz, Latenzverhalten und Stabilität ein.

3.2. Referenzreihe bei 5 bis 100 Usern

Die Referenzreihe bildet den vollständig replizierten Ausgangsbereich der empirischen Evaluation. Sie umfasst die Laststufen 5, 10, 25, 50, 75 und 100 simulierte Locust-User. Jede Laststufe wurde je Datenbanksystem dreimal durchgeführt und anschließend aggregiert.

Die Reihe dient nicht als Grenzbelastungstest, sondern als kontrollierte Basis zur Einordnung des Verhaltens von PostgreSQL und MongoDB bei niedriger bis moderater Parallelität.

Die Auswertung konzentriert sich auf Durchsatz, Median-Latenz, p95-Latenz, p99-Latenz und Fehlerrate. Tabelle 3.2 fasst die aggregierten Gesamtergebnisse der Referenzreihe zusammen.

Tabelle 3.2.: Aggregierte Gesamtergebnisse der Referenzreihe bei 5 bis 100 Usern

System	User	n	Req/s	Median (ms)	p95 (ms)	p99 (ms)	Fail %
MongoDB	5	3	16,10	3,33	27,33	38,67	0,0000
PostgreSQL	5	3	16,22	2,33	17,67	30,33	0,0000
MongoDB	10	3	31,87	3,33	22,33	34,67	0,0000
PostgreSQL	10	3	32,06	2,33	16,33	25,00	0,0000
MongoDB	25	3	80,35	3,00	21,00	31,67	0,0000
PostgreSQL	25	3	80,37	2,67	16,67	25,67	0,0000
MongoDB	50	3	160,80	3,00	25,00	41,33	0,0000
PostgreSQL	50	3	161,85	2,00	15,67	25,33	0,0000
MongoDB	75	3	242,21	3,00	23,00	38,00	0,0000
PostgreSQL	75	3	243,38	2,00	15,33	25,33	0,0000
MongoDB	100	3	318,27	4,67	38,67	84,33	0,0000
PostgreSQL	100	3	324,10	2,33	19,00	38,00	0,0000

Der Durchsatz steigt bei beiden Systemen über die Laststufen hinweg nahezu linear an. Bei 5 Usern erreicht MongoDB im Mittel 16,10 Requests pro Sekunde, PostgreSQL 16,22 Requests pro Sekunde. Bei 100 Usern liegen die Werte bei 318,27 Requests pro Sekunde für MongoDB und 324,10 Requests pro Sekunde für PostgreSQL. Ein Durchsatzeinbruch oder ein erkennbarer Sättigungspunkt tritt in der Referenzreihe nicht auf. Damit zeigen beide Systeme in diesem Lastbereich einen ähnlichen erfolgreichen Gesamtdurchsatz.

Deutlichere Unterschiede zeigen sich im Latenzverhalten. PostgreSQL weist über alle Laststufen hinweg niedrigere Median-, p95- und p99-Latenzen auf. Bei 100 Usern beträgt die Median-Latenz von MongoDB 4,67 ms, während PostgreSQL bei 2,33 ms liegt. Im p95-Bereich erreicht MongoDB bei 100 Usern 38,67 ms, PostgreSQL dagegen 19,00 ms. Noch stärker fällt der Unterschied im p99-Bereich aus: MongoDB erreicht dort 84,33 ms, PostgreSQL 38,00 ms.

Die Referenzreihe zeigt damit bereits vor den Hochlastläufen ein erstes Muster: Der Gesamtdurchsatz beider Systeme liegt bis 100 User eng beieinander, während sich die stärkeren Unterschiede im oberen Latenzbereich zeigen. Dieses Ergebnis beschreibt zunächst den gemessenen Befund der konkreten Versuchsumgebung. Als plausible Einflussfaktoren kommen Unterschiede in Datenmodellierung, Zugriffspfaden, Clientstrategie und transaktionsnahen Operationen in Betracht; eine einzelne Ursache lässt sich aus den aggregierten Referenzwerten jedoch nicht isoliert nachweisen.

In allen gültigen Läufen der Referenzreihe beträgt die aggregierte Fehlerrate 0,0000 %. Es wurden somit weder bei PostgreSQL noch bei MongoDB endgültig fehlgeschlagene Locust-Operationen registriert. Auch die zugehörigen Failure- und Exception-Dateien enthielten für die ausgewerteten Läufe keine relevanten Einträge. Die Systeme unterscheiden sich in dieser Reihe daher nicht über die Fehlerrate, sondern vor allem über die Latenzverteilung.

3.2.1. Operationsbezogene Betrachtung bei 100 Usern

Ergänzend zur aggregierten Betrachtung wird die p95-Latenz der einzelnen Operationstypen bei 100 Usern betrachtet. Diese Laststufe ist die höchste Laststufe der Referenzreihe und zeigt die stärksten Unterschiede im oberen Latenzbereich.

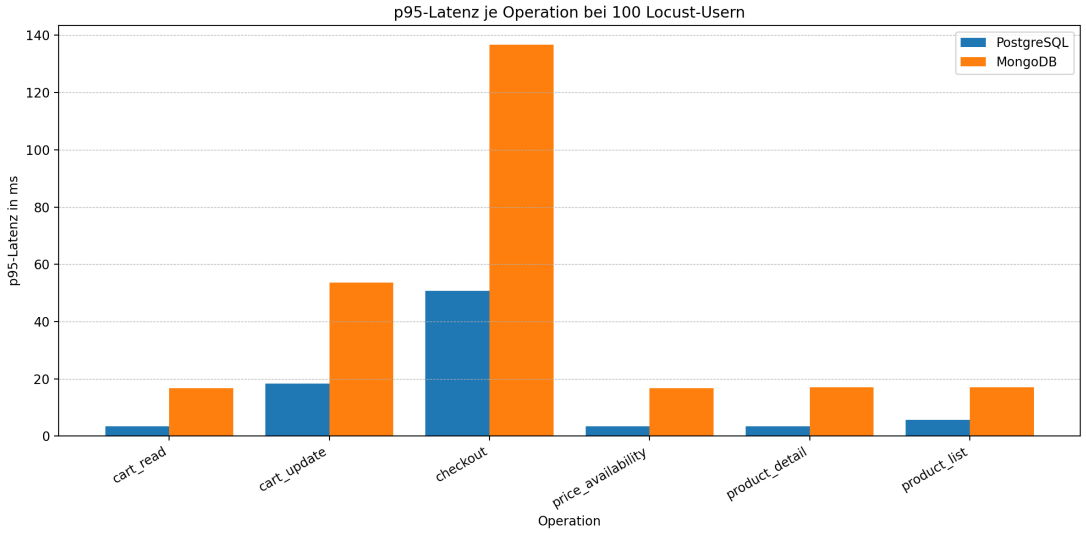


Abbildung 3.1.: p95-Latenz je Operation bei 100 Usern

Abbildung 3.1 zeigt, dass die Unterschiede nicht bei allen Operationen gleich stark auftreten. Lesende Operationen wie `product_detail` oder `price_availability` bleiben vergleichsweise niedrig. Stärkere Unterschiede zeigen sich vor allem bei schreibenden beziehungsweise transaktionsnahen Zugriffsmustern, insbesondere bei `cart_update` und `checkout`. Die checkoutnahe Operation umfasst mehrere zusammenhängende Schritte wie Warenkorbauswahl, Bestandsprüfung, Bestellanlage, Bestandsreduktion und Abschluss des bisherigen Warenkorbs. Dadurch ist sie komplexer als reine Produkt- oder Preisabfragen.

Die operationsbezogene Betrachtung unterstützt die Interpretation, dass die Unterschiede im oberen Latenzbereich vor allem bei schreibenden und checkoutnahen Operationen sichtbar werden. Diese Beobachtung bleibt zunächst auf die Referenzreihe bezogen und wird in den folgenden Hochlastabschnitten erneut aufgegriffen.

3.2.2. Zwischenfazit der Referenzreihe

Die Referenzreihe zeigt, dass PostgreSQL und MongoDB im Bereich von 5 bis 100 Usern einen ähnlichen erfolgreichen Gesamtdurchsatz erreichen. In keiner Laststufe wurden endgültige Failures registriert. Deutlichere Unterschiede zeigen sich im Latenzverhalten: PostgreSQL weist über alle Laststufen hinweg niedrigere Median-, p95- und p99-Latenzen auf, wobei der Abstand besonders im p99-Bereich bei 100 Usern sichtbar wird.

Für den roten Faden der Evaluation bedeutet dies: Die Referenzreihe liefert einen stabilen und replizierten Ausgangspunkt, erreicht aber noch keinen deutlich degradierten Hochlast- oder Grenzlastbereich. Die zentrale Lastbetrachtung erfolgt deshalb in den folgenden Abschnitten anhand der replizierten Hochlastläufe bei 500 und 1.000 Usern sowie der explorativen Grenzlastläufe bei 2.500 und 5.000 Usern.

3.3. Replizierter Hochlastbereich bei 500 und 1.000 Usern

Der replizierte Hochlastbereich bildet den zentralen Lastbereich der empirischen Evaluation. Im Unterschied zur Referenzreihe bei 5 bis 100 Usern wird hier untersucht, wie sich PostgreSQL und MongoDB bei deutlich höherer Parallelität verhalten. Die Laststufen 500 und 1.000 User wurden je Datenbanksystem dreimal durchgeführt und aggregiert. Damit lassen sich in diesem Bereich wiederholte Messwerte statt einzelner Beobachtungen vergleichen.

Tabelle 3.3 fasst die aggregierten Gesamtergebnisse des replizierten Hochlastbereichs zusammen. Die Werte zeigen einen deutlichen Übergang gegenüber der Referenzreihe: Bei 500 Usern liegen die Systeme noch vergleichsweise nah beieinander, während bei 1.000 Usern stärkere Unterschiede in Durchsatz und Tail-Latenzen sichtbar werden.

Tabelle 3.3.: Aggregierte Gesamtergebnisse des replizierten Hochlastbereichs

System	User	n	Req/s	Median (ms)	p95 (ms)	p99 (ms)	Fail %
PostgreSQL	500	3	632,35	220,00	1.466,67	3.700,00	0,0000
MongoDB	500	3	559,46	390,00	1.500,00	2.766,67	0,0002
PostgreSQL	1.000	3	393,39	973,33	8.300,00	17.000,00	0,0000
MongoDB	1.000	3	513,11	1.100,00	4.600,00	7.700,00	0,0013

3.3.1. Durchsatz und Median-Latenz

Abbildung 3.2 zeigt den Durchsatz der ergänzenden Hoch- und Grenzlastreihe. Für den vorliegenden Abschnitt sind die replizierten Laststufen 500 und 1.000 User relevant; die rechts der gestrichelten Trennlinie liegenden Grenzlaststufen werden in Abschnitt 3.4 gesondert betrachtet. Die gestrichelte Linie markiert in den folgenden Hochlast- und Grenzlastdiagrammen den Übergang vom replizierten Hochlastbereich zum explorativen Grenzlastbereich. Die methodische Besonderheit der Grenzlastwerte wird in Abschnitt 3.4 und Tabelle 3.4 erläutert.

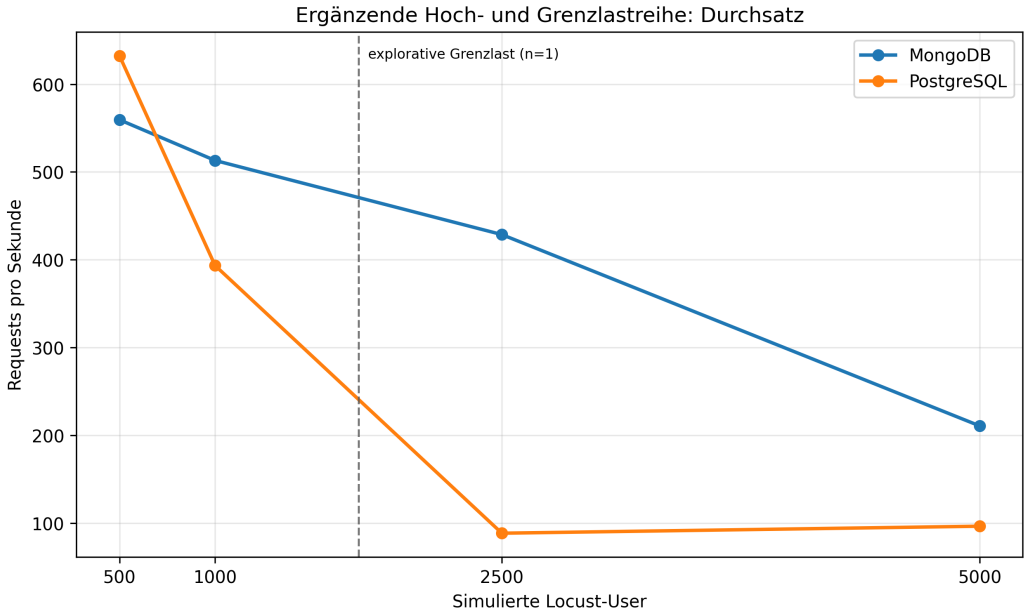


Abbildung 3.2.: Durchsatz der ergänzenden Hoch- und Grenzlastreihe

Bei 500 Usern erreicht PostgreSQL mit 632,35 Requests pro Sekunde einen höheren aggregierten Durchsatz als MongoDB mit 559,46 Requests pro Sekunde. Gleichzeitig liegt PostgreSQL mit einer Median-Latenz von 220,00 ms unter dem MongoDB-Wert von 390,00 ms. In dieser Laststufe verarbeitet PostgreSQL das Lastprofil somit mit höherem Durchsatz und niedrigerer typischer Antwortzeit.

Bei 1.000 Usern verändert sich das Bild. PostgreSQL fällt auf 393,39 Requests pro Sekunde zurück, während MongoDB 513,11 Requests pro Sekunde erreicht. Gemessen an den aggregierten Durchsatzwerten verarbeitet MongoDB in dieser Laststufe rund 30 % mehr Requests pro Sekunde als PostgreSQL. Der Rückgang von PostgreSQL zwischen 500 und 1.000 Usern ist damit deutlich stärker als bei MongoDB.

Abbildung 3.3 zeigt ergänzend die Median-Latenzen der Hoch- und Grenzlastreihe. Im replizierten Hochlastbereich steigen die Median-Werte beider Systeme gegenüber der Referenzreihe deutlich an.

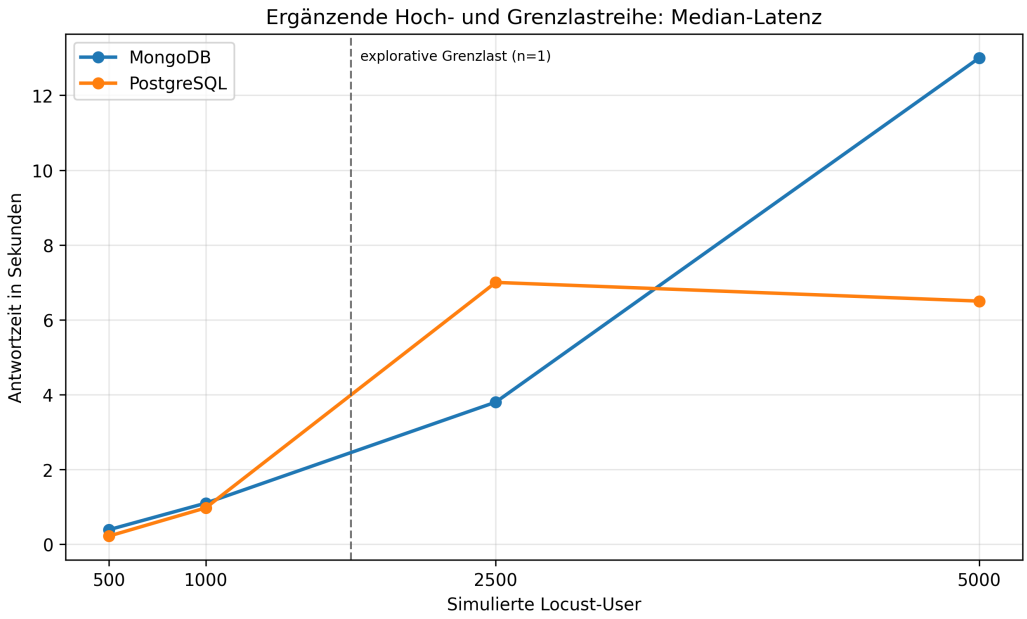


Abbildung 3.3.: Median-Latenz der ergänzenden Hoch- und Grenzlastreihe

PostgreSQL bleibt im Median auch bei 1.000 Usern geringfügig niedriger als MongoDB. Bei 500 Usern liegen die Median-Latenzen bei 220,00 ms für PostgreSQL und 390,00 ms für MongoDB. Bei 1.000 Usern steigen sie auf 973,33 ms beziehungsweise 1.100,00 ms. Damit zeigt PostgreSQL im typischen Antwortzeitbereich weiterhin kürzere Werte, verliert bei 1.000 Usern aber gleichzeitig deutlich an Durchsatz.

3.3.2. Tail-Latenzen und Fehlerrate

Für die Bewertung unter hoher Last sind die oberen Latenzbereiche besonders wichtig, weil sie zeigen, wie stark einzelne erfolgreiche Operationen vom typischen Antwortzeitverhalten abweichen. Abbildung 3.4 und Abbildung 3.5 zeigen die p95- und p99-Latenzen der ergänzenden Hoch- und Grenzlastreihe.

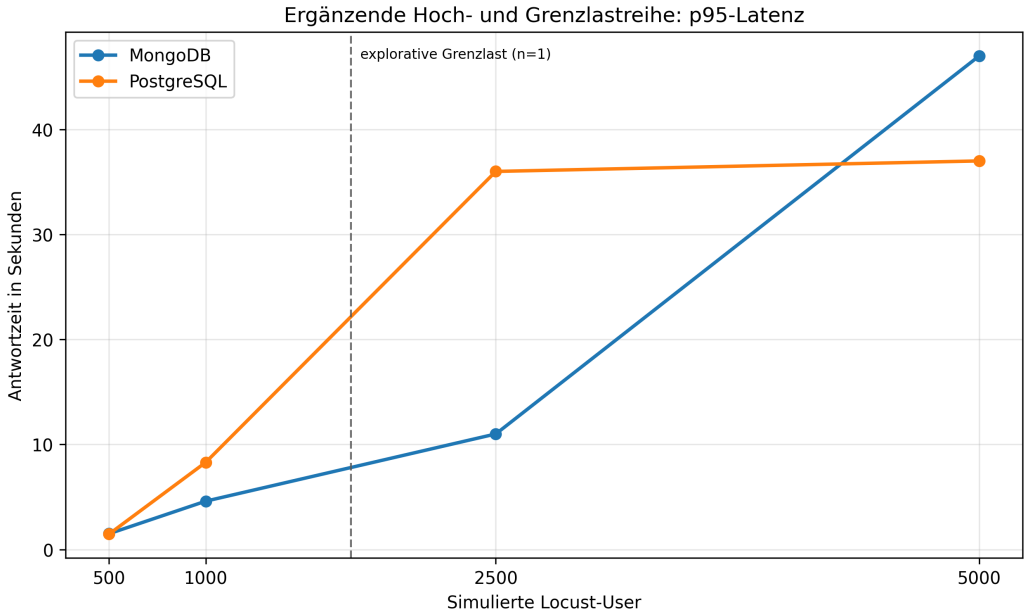


Abbildung 3.4.: p95-Latenz der ergänzenden Hoch- und Grenzlastreihe

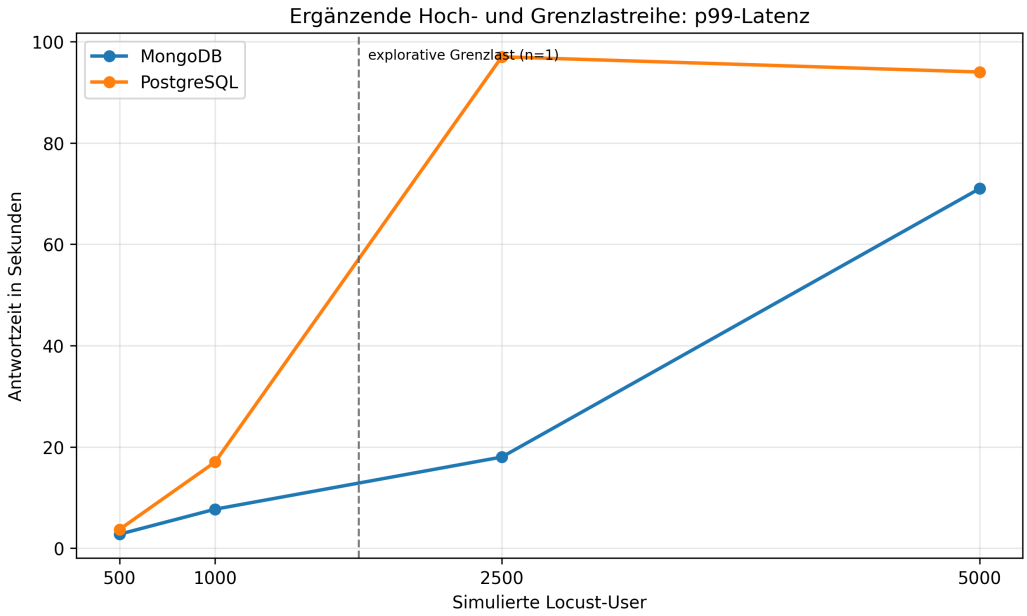


Abbildung 3.5.: p99-Latenz der ergänzenden Hoch- und Grenzlastreihe

Bei 500 Usern liegen die p95-Werte beider Systeme noch nah beieinander: PostgreSQL erreicht 1.466,67 ms, MongoDB 1.500,00 ms. Im p99-Bereich liegt MongoDB mit 2.766,67 ms niedriger als PostgreSQL mit 3.700,00 ms. Die Laststufe 500 zeigt damit kein einheitliches Bild zugunsten eines Systems: PostgreSQL weist den höheren Durchsatz und die niedrigere Median-Latenz auf, während MongoDB den niedrigeren p99-Wert erreicht.

Bei 1.000 Usern wird der Unterschied im oberen Latenzbereich deutlicher. PostgreSQL erreicht eine p95-Latenz von 8.300,00 ms und eine p99-Latenz von 17.000,00 ms. MongoDB liegt in derselben Laststufe bei 4.600,00 ms im p95-Bereich und 7.700,00 ms im p99-Bereich. Damit ist die obere Latenzverteilung von PostgreSQL bei 1.000 Usern deutlich stärker ausgeprägt als bei MongoDB.

Abbildung 3.6 zeigt die Fehlerraten der ergänzenden Hoch- und Grenzlastreihe. Im replizierten Hochlastbereich bleiben die Fehlerraten insgesamt sehr niedrig.

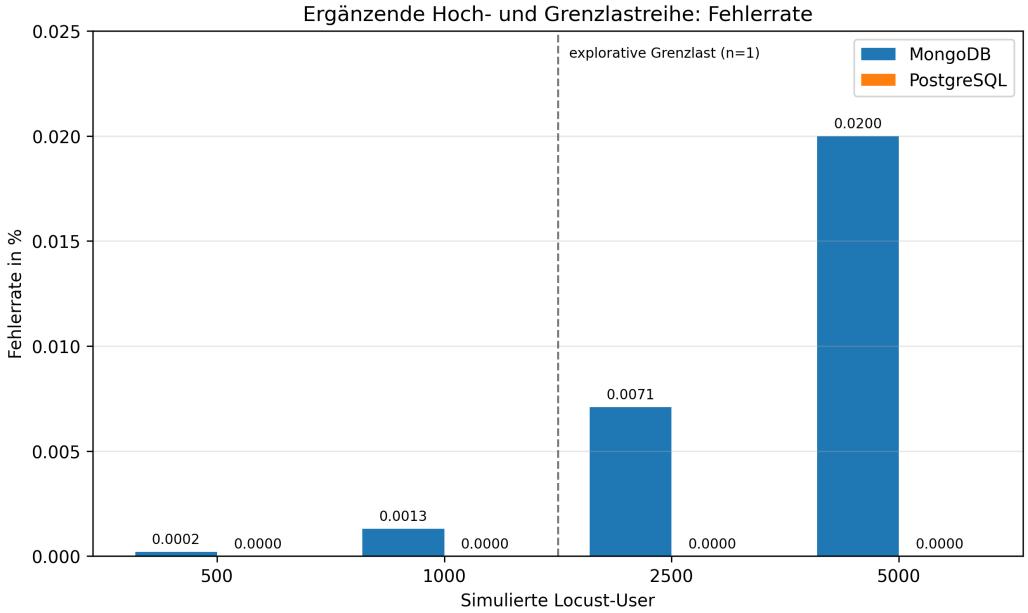


Abbildung 3.6.: Fehlerrate der ergänzenden Hoch- und Grenzlastreihe

PostgreSQL weist bei 500 und 1.000 Usern eine aggregierte Fehlerrate von 0,0000 % auf. Für MongoDB liegt die Fehlerrate bei 500 Usern bei 0,0002 % und bei 1.000 Usern bei 0,0013 %. Die MongoDB-Fehlerraten sind damit messbar, bleiben im Verhältnis zur Gesamtzahl der Requests aber sehr niedrig. Die Systemunterschiede werden in diesem Bereich daher nicht primär über endgültige Fehler sichtbar, sondern vor allem über Durchsatz und Latenzverteilung.

3.3.3. Zwischenfazit des replizierten Hochlastbereichs

Der replizierte Hochlastbereich zeigt einen klaren Übergang gegenüber der Referenzreihe. Bei 500 Usern erreicht PostgreSQL den höheren Durchsatz und die niedrigere Median-Latenz, während die oberen Latenzbereiche beider Systeme noch relativ nah beieinander liegen. Bei 1.000 Usern verändert sich das Bild: MongoDB erreicht den höheren Durchsatz und deutlich niedrigere p95- und p99-Latenzen, während PostgreSQL formal fehlerfrei bleibt, aber stärker degradiert.

Als gemessener Befund lässt sich festhalten, dass PostgreSQL im dokumentierten Versuchsaufbau bei 500 Usern günstiger abschneidet, bei 1.000 Usern jedoch einen deutlichen Rückgang im Durchsatz und stark steigende Tail-Latenzen zeigt. MongoDB zeigt von 500 auf 1.000 User ebenfalls steigende Latenzen, hält den Durchsatz aber stabiler und weist bei 1.000 Usern geringere Tail-Latenzen auf. Die sehr niedrigen MongoDB-Fehlerraten sprechen nicht für einen allgemeinen Stabilitätsverlust, zeigen aber, dass unter hoher konkurrierender Last einzelne Operationen endgültig fehlschlagen können.

Für den roten Faden der Evaluation ist dieser Abschnitt der erste zentrale Hochlastbefund: Die Unterschiede zwischen den Systemen treten nicht bereits in der Referenzreihe, sondern vor allem bei steigender Parallelität hervor. Die anschließende Grenzlastbetrachtung untersucht, wie sich dieses Verhalten bei noch stärkeren Laststufen weiterentwickelt.

3.4. Explorative Grenzlast bei 2.500 und 5.000 Usern

Nach dem replizierten Hochlastbereich wurden ergänzend Grenzlastläufe bei 2.500 und 5.000 simulierten Locust-Usern durchgeführt. Diese Läufe zeigen, wie sich die lokale Versuchsumgebung in einem deutlich stärker belasteten Bereich verhält. Im Unterschied zu den Laststufen 500 und 1.000 User wurden die Grenzlaststufen je System nur einmal durchgeführt. Die Werte werden deshalb als explorative Einzelmessungen ohne Streuungsaussage interpretiert.

Für PostgreSQL wurden die Grenzlastwerte auf Basis des vollständigen Messlaufs ausgewertet. Für MongoDB wurde bei 2.500 und 5.000 Usern die stabile Zieluserphase als Auswertungsfenster verwendet, da nach Ende der eigentlichen zehnminütigen Zielphase noch lange offene Requests zurückliefen. Dieser methodische Unterschied ist für die Interpretation wesentlich: Die Messwerte zeigen das Verhalten im dokumentierten Grenzbereich, bilden aber keine vollständig symmetrische Wiederholungsreihe.

Tabelle 3.4 fasst die gemessenen Grenzlastwerte zusammen.

Tabelle 3.4.: Ergebnisse der explorativen Grenzlastläufe bei 2.500 und 5.000 Usern. PostgreSQL wurde auf Basis des vollständigen Messlaufs ausgewertet; MongoDB auf Basis der stabilen Zieluserphase.

System	User	n	Req/s	Median (ms)	p95 (ms)	p99 (ms)	Fail %
PostgreSQL	2.500	1	88,42	7.000,00	36.000,00	97.000,00	0,0000
MongoDB	2.500	1	428,66	3.800,00	11.000,00	18.000,00	0,0071
PostgreSQL	5.000	1	96,39	6.500,00	37.000,00	94.000,00	0,0000
MongoDB	5.000	1	210,64	13.000,00	47.000,00	71.000,00	0,0200

3.4.1. Durchsatz und Latenzverhalten in der Grenzlast

Der Durchsatz der Grenzlaststufen ist in Abbildung 3.2 gemeinsam mit dem replizierten Hochlastbereich dargestellt. Die gestrichelte Trennlinie markiert den Übergang von den replizierten Hochlaststufen zu den explorativen Grenzlaststufen.

Bei PostgreSQL fällt der Durchsatz von 393,39 Requests pro Sekunde bei 1.000 Usern auf 88,42 Requests pro Sekunde bei 2.500 Usern. Bei 5.000 Usern liegt der Wert mit 96,39 Requests pro Sekunde in einer ähnlichen Größenordnung. Die leichte Erhöhung gegenüber 2.500 Usern wird wegen der Einzelmessung nicht als belastbarer Skalierungseffekt interpretiert. Entscheidend ist, dass PostgreSQL in beiden Grenzlaststufen nur noch einen deutlich geringeren Durchsatz erreicht als im replizierten Hochlastbereich.

MongoDB erreicht bei 2.500 Usern 428,66 Requests pro Sekunde und liegt damit noch in der Nähe des replizierten 1.000-User-Bereichs. Bei 5.000 Usern fällt der Durchsatz auf 210,64 Requests pro Sekunde. Damit zeigt auch MongoDB im Grenzlastbereich eine deutliche Degradation, allerdings später und weniger abrupt als PostgreSQL im Übergang von 1.000 auf 2.500 User.

Die Median-, p95- und p99-Latenzen sind in den Abbildungen 3.3, 3.4 und 3.5 dargestellt. Bei PostgreSQL erreicht die Median-Latenz 7.000,00 ms bei 2.500 Usern und 6.500,00 ms bei 5.000 Usern. Die p95-Latenzen liegen bei 36.000,00 ms beziehungsweise 37.000,00 ms, die p99-Latenzen bei 97.000,00 ms beziehungsweise 94.000,00 ms. PostgreSQL verarbeitet in diesem Bereich also weiterhin erfolgreiche Operationen, diese sind jedoch mit sehr langen Wartezeiten verbunden.

MongoDB zeigt bei 2.500 Usern mit 3.800,00 ms Median-Latenz, 11.000,00 ms p95-Latenz und 18.000,00 ms p99-Latenz zunächst deutlich niedrigere Werte als PostgreSQL. Bei 5.000 Usern verschlechtert sich das Bild jedoch stark: Der Median steigt auf 13.000,00 ms, p95 auf 47.000,00 ms und p99 auf 71.000,00 ms. Der Grenzbereich wird damit bei beiden Systemen sichtbar, aber nicht in derselben Laststufe und nicht mit identischem Verlauf.

3.4.2. Fehlerrate und Stabilität in der Grenzlast

Die Fehlerraten der Grenzlaststufen sind in Abbildung 3.6 dargestellt. PostgreSQL weist auch bei 2.500 und 5.000 Usern eine Fehlerrate von 0,0000 % auf. Dieses Ergebnis zeigt, dass im jeweiligen Messlauf keine endgültig fehlgeschlagenen Locust-Operationen registriert wurden. Es bedeutet jedoch nicht, dass die Verarbeitung unter dieser Last performant blieb. Die niedrigen Durchsatzwerte und die hohen Tail-Latenzen zeigen vielmehr eine starke Verzögerung erfolgreicher Operationen.

MongoDB zeigt bei 2.500 Usern eine Fehlerrate von 0,0071 % und bei 5.000 Usern eine Fehlerrate von 0,0200 %. Die absoluten Fehlerraten bleiben damit niedrig, steigen aber gegenüber dem replizierten Hochlastbereich sichtbar an. Die endgültigen Fehler betreffen checkoutnahe Konfliktsituationen, die nach den begrenzten Wiederholungsversuchen nicht erfolgreich abgeschlossen wurden.

Für die Stabilitätsbewertung bedeutet dies: PostgreSQL bleibt in der Grenzlast formal fehlerfrei, zeigt aber eine starke Leistungsdegradation über Durchsatzrückgang und sehr hohe Wartezeiten. MongoDB verarbeitet im Grenzlastbereich mehr Requests als PostgreSQL, erzeugt aber einzelne endgültige Fehler. Die Fehlerrate allein reicht daher nicht aus, um Stabilität zu bewerten; sie muss gemeinsam mit Durchsatz, Median-Latenz und Tail-Latenzen betrachtet werden.

3.4.3. Zwischenfazit der explorativen Grenzlast

Die explorativen Grenzlastläufe zeigen den Übergang von hoher Last zu einem stark degradierten Lastbereich. PostgreSQL erreicht bei 2.500 und 5.000 Usern nur noch einen geringen Durchsatz und weist sehr hohe p95- und p99-Latenzen auf. Obwohl keine endgültigen Failures registriert wurden, ist die Verarbeitung in diesem Bereich mit sehr langen Antwortzeiten verbunden.

MongoDB erreicht bei 2.500 Usern noch einen deutlich höheren Durchsatz und niedrigere Tail-Latenzen als PostgreSQL. Bei 5.000 Usern sinkt jedoch auch bei MongoDB der Durchsatz stark, während Median- und Tail-Latenzen deutlich steigen. Zusätzlich treten einzelne endgültige Failures auf. Der Grenzlastbereich zeigt damit keinen einfachen Systemvorteil, sondern unterschiedliche Degradationsmuster: PostgreSQL degradiert in dieser lokalen Versuchsumgebung vor allem über Durchsatzrückgang und sehr hohe Wartezeiten, MongoDB über eine später einsetzende, aber bei 5.000 Usern ebenfalls deutliche Latenz- und Durchsatzdegradation mit sehr niedriger, aber vorhandener Fehlerrate.

Für die Forschungsfragen ist dieser Abschnitt wichtig, weil er erstmals den Bereich sichtbar macht, in dem die Systeme nicht mehr nur unterschiedlich schnell reagieren, sondern in einen deutlich überlasteten Betriebszustand übergehen. Die nachfolgende Datenmengen-Sensitivitätsanalyse betrachtet anschließend nicht zusätzliche Parallelität, sondern den Einfluss eines größeren Produktkatalogs bei konstanter Laststufe.

3.5. Datenmengen-Sensitivitätsanalyse

Neben der Erhöhung der parallelen Userzahl wurde ergänzend untersucht, wie sich ein größerer Produktkatalog auf die Messergebnisse auswirkt. Dafür wurde bei konstanter Laststufe von 500 simulierten Locust-Usern das Profil `medium` mit dem Profil `large_catalog` verglichen. Das Profil `medium` enthält 50.000 Produkte und Bestandsdatensätze, während `large_catalog` 200.000 Produkte und Bestandsdatensätze enthält. Die Zahl der Kunden und aktiven Warenkörbe bleibt in beiden Profilen bei 5.000.

Diese Versuchsanordnung betrachtet nicht zusätzliche Parallelität, sondern den Einfluss einer größeren Produkt- und Bestandsmenge bei gleichbleibender Nutzerzahl. Beide Datenbanksysteme wurden für diese Analyse jeweils dreimal pro Datensatzprofil gemessen und anschließend aggregiert. Tabelle 3.5 fasst die zentralen Vergleichswerte zusammen.

Tabelle 3.5.: Datenmengen-Sensitivität bei 500 Usern

System	Metrik	`medium`	`large_catalog`	Veränderung
PostgreSQL	Durchsatz (Req/s)	632,35	602,97	-4,65 %
PostgreSQL	Median (ms)	220,00	246,67	+12,12 %
PostgreSQL	p95 (ms)	1.466,67	1.600,00	+9,09 %
PostgreSQL	p99 (ms)	3.700,00	3.800,00	+2,70 %
MongoDB	Durchsatz (Req/s)	559,46	293,26	-47,58 %
MongoDB	Median (ms)	390,00	930,00	+138,46 %
MongoDB	p95 (ms)	1.500,00	3.933,33	+162,22 %
MongoDB	p99 (ms)	2.766,67	6.533,33	+136,14 %

3.5.1. Durchsatz und Median-Latenz bei größerem Produktkatalog

Abbildung 3.7 zeigt den aggregierten Durchsatz bei 500 Usern für beide Datensatzprofile. Bei PostgreSQL sinkt der Durchsatz von 632,35 Requests pro Sekunde im Profil `medium` auf 602,97 Requests pro Sekunde im Profil `large_catalog`. Das entspricht einem Rückgang von 4,65 %.

Bei MongoDB fällt der Durchsatz deutlich stärker. Im Profil `medium` erreicht MongoDB 559,46 Requests pro Sekunde, im Profil `large_catalog` dagegen 293,26 Requests pro Sekunde. Das entspricht einem Rückgang von 47,58 %. Der gemessene Befund zeigt damit, dass die Vergrößerung des Produktkatalogs in dieser Versuchsanordnung bei MongoDB mit einem wesentlich stärkeren Durchsatzverlust verbunden ist als bei PostgreSQL.

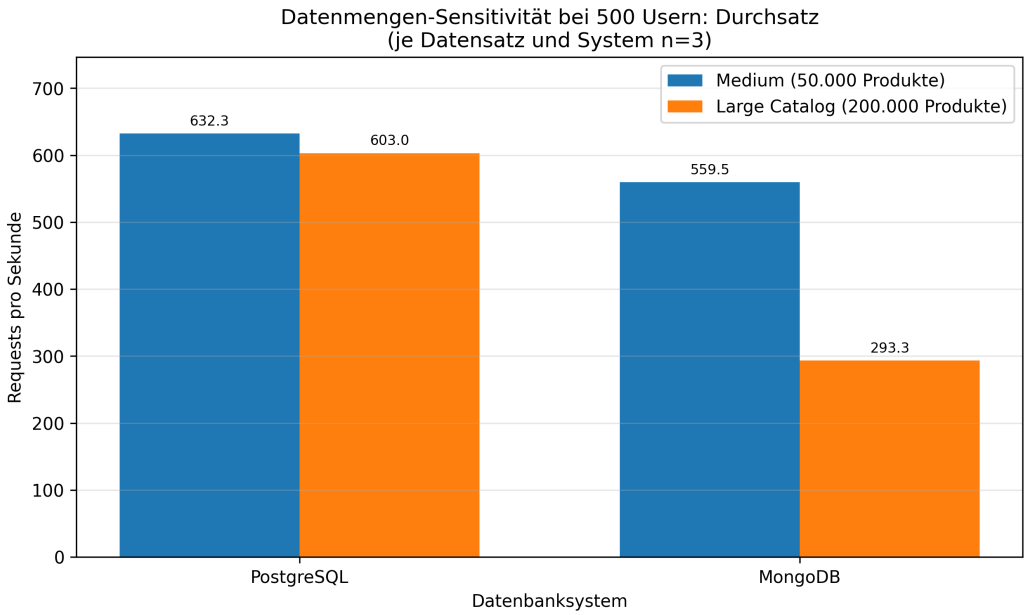


Abbildung 3.7.: Datenmengen-Sensitivität bei 500 Usern: Durchsatz

Abbildung 3.8 zeigt ergänzend die Median-Latenzen. Bei PostgreSQL steigt der Median von 220,00 ms auf 246,67 ms. Die relative Veränderung beträgt 12,12 %. Damit bleibt die typische Antwortzeit trotz größerem Produktkatalog in einer ähnlichen Größenordnung wie im Profil `medium`.

Bei MongoDB steigt die Median-Latenz von 390,00 ms auf 930,00 ms. Dies entspricht einer Erhöhung um 138,46 %. Die typische Antwortzeit verschiebt sich damit deutlich stärker als bei PostgreSQL. Zusammen mit dem Durchsatzrückgang zeigt dies, dass MongoDB im Profil `large_catalog` weniger Requests pro Sekunde verarbeitet und im Median länger pro erfolgreicher Operation benötigt.

3.5.2. Tail-Latenzen bei größerem Produktkatalog

Für die Bewertung der Datenmengen-Sensitivität sind neben dem Median auch die oberen Latenzbereiche relevant. Abbildung 3.9 zeigt die p95-Latenzen. Bei PostgreSQL steigt die p95-Latenz von 1.466,67 ms auf 1.600,00 ms. Das entspricht einer relativen Erhöhung von 9,09 %.

Bei MongoDB steigt die p95-Latenz von 1.500,00 ms auf 3.933,33 ms. Die relative Erhöhung beträgt 162,22 %. Damit zeigt sich im oberen Latenzbereich ein noch deutlicherer Datenmengeneffekt als im Median.

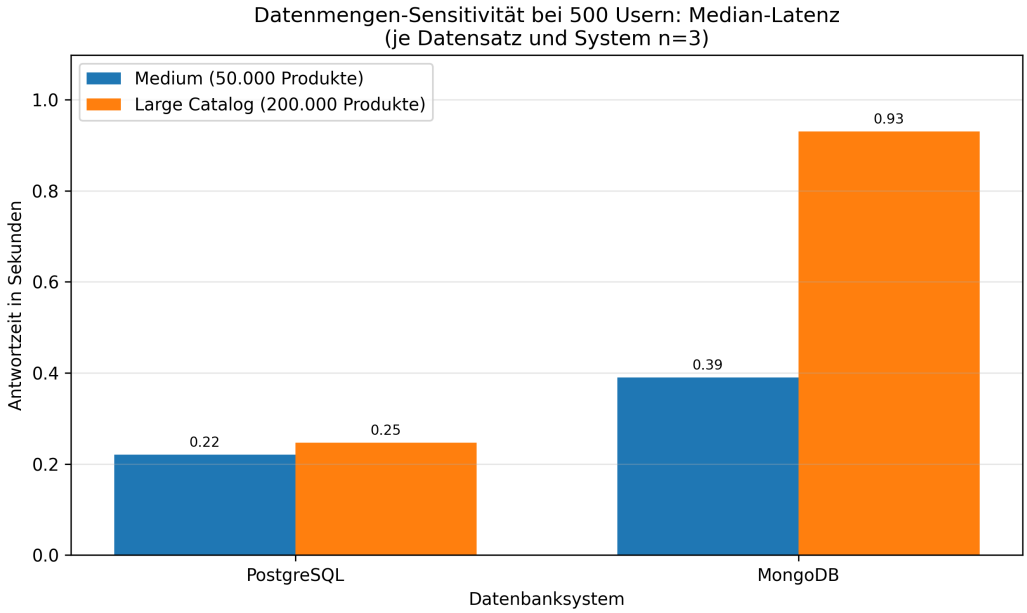


Abbildung 3.8.: Datenmengen-Sensitivität bei 500 Usern: Median-Latenz

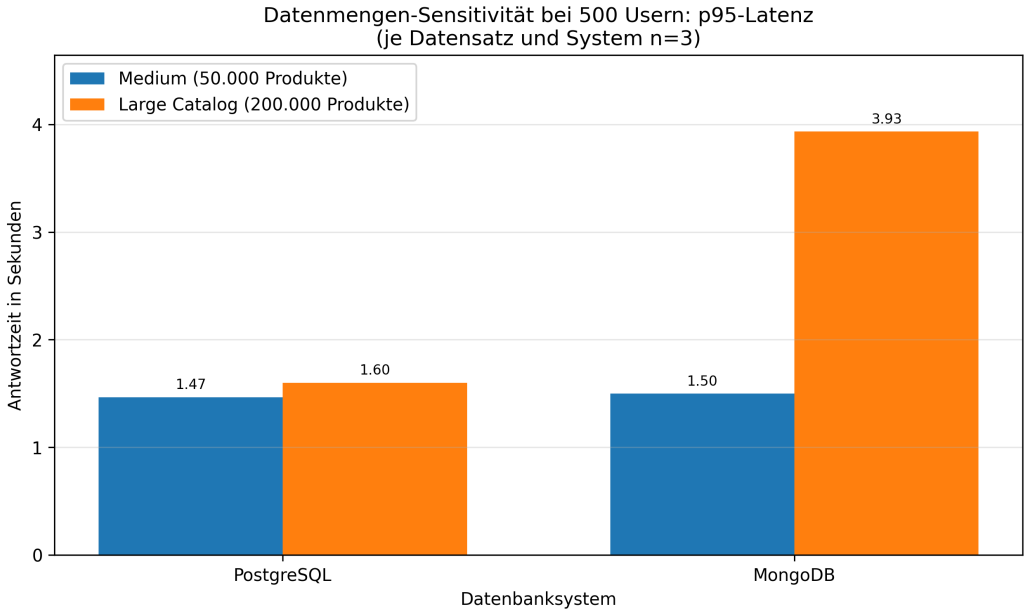


Abbildung 3.9.: Datenmengen-Sensitivität bei 500 Usern: p95-Latenz

Abbildung 3.10 zeigt die p99-Latenzen. Bei PostgreSQL steigt der p99-Wert nur leicht von 3.700,00 ms auf 3.800,00 ms. Die relative Veränderung beträgt 2,70 %. Bei MongoDB steigt der p99-Wert von 2.766,67 ms auf 6.533,33 ms und damit um 136,14 %.

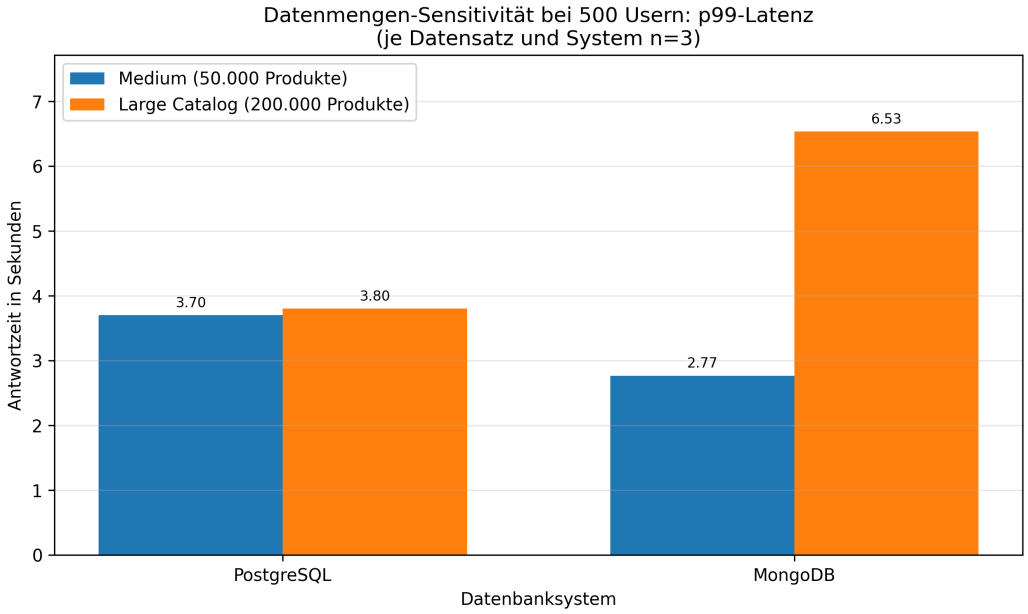


Abbildung 3.10.: Datenmengen-Sensitivität bei 500 Usern: p99-Latenz

Die Tail-Latenzen bestätigen den Befund aus Durchsatz und Median-Latenz. PostgreSQL reagiert im dokumentierten Aufbau vergleichsweise moderat auf die Erhöhung des Produktkatalogs. MongoDB zeigt dagegen bei konstant 500 Usern eine deutliche Verschlechterung sowohl im typischen Antwortzeitbereich als auch in den oberen Latenzbereichen.

3.5.3. Stabilität und Zwischenfazit der Datenmengen-Sensitivitätsanalyse

In den gültigen Läufen der Datenmengen-Sensitivitätsanalyse wurden keine endgültigen Failures oder Exceptions festgestellt. Damit unterscheiden sich die beiden Systeme in diesem Auswertungsbereich nicht über die Fehlerrate. Die zentrale Differenz liegt vielmehr in der Leistungswirkung des größeren Produktkatalogs.

Für die Einordnung ist wichtig, dass die Datenmengen-Sensitivitätsanalyse keine neue Grenzlaststufe darstellt. Die Userzahl bleibt konstant bei 500 Usern. Der beobachtete Effekt entsteht daher nicht durch mehr simulierte gleichzeitige Nutzer, sondern durch die veränderte Datengrundlage. Gerade deshalb ergänzt diese Analyse die Hochlast- und Grenzlastbetrachtung: Sie zeigt, dass nicht nur Parallelität, sondern auch die Größe des Produktkatalogs das gemessene Systemverhalten beeinflussen kann.

Als gemessener Befund lässt sich festhalten, dass PostgreSQL beim Wechsel von `medium` zu `large_catalog` nur moderate Veränderungen aufweist. Der Durchsatz sinkt um 4,65 %, während Median-, p95- und p99-Latenzen vergleichsweise gering steigen. MongoDB reagiert deutlich stärker auf die Vergrößerung des Produktkatalogs: Der Durchsatz sinkt um 47,58 %, während Median-, p95- und p99-Latenzen jeweils deutlich ansteigen.

Die Ergebnisse zeigen damit, dass der größere Produktkatalog im dokumentierten Versuchsaufbau bei PostgreSQL mit geringeren relativen Veränderungen verbunden war als bei MongoDB. Diese Aussage gilt für das hier verwendete E-Commerce-nahe Lastprofil, die konkrete Datenmodellierung und die lokale Docker-basierte Versuchsumgebung. Eine allgemeine Aussage über alle relationalen oder dokumentenorientierten Datenbanksysteme lässt sich daraus nicht ableiten.

3.6. Zusammenführende Ergebnisbewertung

Die empirische Evaluation zeigt, dass sich das Verhalten von PostgreSQL und MongoDB im untersuchten Versuchsaufbau nicht durch eine einzelne Kennzahl ausreichend beschreiben lässt. Der Vergleich muss gemeinsam über Durchsatz, Median-Latenz, obere Latenzbereiche und Fehlerraten erfolgen. Zudem hängt die Bewertung deutlich vom betrachteten Last- beziehungsweise Datensatzbereich ab.

In der Referenzreihe bei 5 bis 100 Usern erreichen PostgreSQL und MongoDB einen ähnlichen erfolgreichen Gesamtdurchsatz. Beide Systeme verarbeiten das definierte E-Commerce-nahe Lastprofil in diesem Bereich ohne endgültige Failures. Unterschiede zeigen sich vor allem im Latenzverhalten: PostgreSQL weist niedrigere Median-, p95- und p99-Latenzen auf, während der Durchsatz beider Systeme eng beieinanderliegt. Die Referenzreihe liefert damit einen stabilen Ausgangspunkt, bildet aber noch keinen deutlich degradierten Hochlastbereich ab.

Im replizierten Hochlastbereich bei 500 und 1.000 Usern werden die Unterschiede deutlicher. Bei 500 Usern erreicht PostgreSQL den höheren Durchsatz und die niedrigere Median-Latenz. Bei 1.000 Usern verändert sich das Bild: MongoDB erreicht den höheren Durchsatz und deutlich niedrigere p95- und p99-Latenzen, während PostgreSQL formal fehlerfrei bleibt, aber einen deutlichen Durchsatzrückgang und stark erhöhte Tail-Latenzen zeigt. Damit wird sichtbar, dass zusätzliche Parallelität nicht in allen Lastbereichen linear in höheren Durchsatz umgesetzt wird.

Die explorativen Grenzlastläufe bei 2.500 und 5.000 Usern zeigen für beide Systeme einen deutlich degradierten Betriebsbereich. PostgreSQL weist in diesen Laststufen sehr niedrige Durchsatzwerte und sehr hohe Tail-Latenzen auf, obwohl keine endgültigen Failures registriert wurden. MongoDB erreicht bei 2.500 Usern noch einen deutlich höheren Durchsatz und niedrigere Tail-Latenzen als PostgreSQL, degradiert bei 5.000 Usern jedoch ebenfalls stark. Zusätzlich treten bei MongoDB einzelne endgültige Failures auf. Die Grenzlastläufe zeigen damit unterschiedliche Degradationsmuster: PostgreSQL degradiert in dieser Versuchsumgebung vor allem über lange Wartezeiten und geringen Durchsatz, MongoDB über steigende Latenzen, sinkenden Durchsatz und eine sehr niedrige, aber vorhandene Fehlerrate.

Die Datenmengen-Sensitivitätsanalyse ergänzt diese Lastbetrachtung um den Einfluss eines größeren Produktkatalogs. Bei konstanter Laststufe von 500 Usern reagiert PostgreSQL nur moderat auf den Wechsel von `medium` zu `large_catalog`. MongoDB zeigt dagegen einen deutlichen Rückgang des Durchsatzes sowie stark steigende Median-, p95- und p99-Latenzen. Damit wird sichtbar, dass die gemessene Leistungsfähigkeit nicht nur von der Anzahl paralleler Nutzer, sondern auch von der Größe der zugrunde liegenden Produkt- und Bestandsdaten beeinflusst wird.

Zusammenfassend ergeben sich drei zentrale Ergebnislinien. Erstens sind PostgreSQL und MongoDB im niedrigen bis moderaten Lastbereich hinsichtlich des Durchsatzes ähnlich, unterscheiden sich aber im Latenzverhalten. Zweitens verschiebt sich das Verhalten unter hoher Last: PostgreSQL zeigt bei 500 Usern günstigere Durchsatz- und Medianwerte, während MongoDB bei 1.000 Usern höhere Durchsatzwerte und niedrigere obere Latenzen erreicht. Drittens ist der größere Produktkatalog im dokumentierten Versuchsaufbau bei PostgreSQL mit geringeren relativen Veränderungen der Leistungskennzahlen verbunden als bei MongoDB.

Diese Ergebnisse gelten für den dokumentierten Versuchsaufbau mit synthetischer Datengrundlage, direktem Datenbankzugriff, definiertem E-Commerce-nahem Lastprofil und lokaler Docker-basierter Testumgebung. Sie zeigen keine pauschale Überlegenheit eines Datenbanktyps, sondern beschreiben das Verhalten von PostgreSQL und MongoDB unter den kontrollierten Bedingungen dieser Arbeit. Die abschließende Beantwortung der Forschungsfragen und die Einordnung der methodischen Grenzen erfolgen in Kapitel 4.

4. Schlussbetrachtung

Dieses Kapitel führt die Ergebnisse der Arbeit zusammen und beantwortet die in Kapitel 1 formulierten Forschungsfragen. Die Schlussbetrachtung verdichtet die zentralen Befunde entlang der Untersuchungsbereiche Referenzreihe, replizierter Hochlastbereich, explorative Grenzlast und Datenmengen-Sensitivitätsanalyse. Anschließend werden die methodischen Grenzen eingeordnet, Ansatzpunkte für weiterführende Arbeiten dargestellt und die zentrale Aussage der Arbeit zusammengefasst.

4.1. Beantwortung der Forschungsfragen

Die Forschungsfragen werden im Folgenden zusammenfassend auf Grundlage der empirischen Ergebnisse aus Kapitel 3 beantwortet. Dabei werden die Messwerte nicht erneut vollständig dargestellt, sondern entlang der zentralen Untersuchungsbereiche eingeordnet.

Forschungsfrage F1: Referenzverhalten.

Die Referenzreihe zeigt, dass PostgreSQL und MongoDB im kontrollierten Referenzbereich grundsätzlich stabil arbeiteten. In den betrachteten Referenzläufen traten keine endgültigen Failures oder Exceptions auf. Der Durchsatz stieg bei beiden Systemen mit zunehmender Nutzerzahl deutlich an und lag bei der höchsten Referenzlast auf einem ähnlichen Niveau. Bei 100 simulierten Usern erreichte PostgreSQL im Mittel 324,10 Requests pro Sekunde, MongoDB 318,27 Requests pro Sekunde.

Unterschiede zeigten sich vor allem im Latenzverhalten. PostgreSQL wies im Referenzbereich insgesamt niedrigere Median-, p95- und p99-Latenzen auf. Bei 100 simulierten Usern lag PostgreSQL bei einer Median-Latenz von 2,33 ms, einer p95-Latenz von 19,00 ms und einer p99-Latenz von 38,00 ms. MongoDB erreichte bei derselben Laststufe eine Median-Latenz von 4,67 ms, eine p95-Latenz von 38,67 ms und eine p99-Latenz von 84,33 ms. Damit war im Referenzbereich nicht die Fehlerrate oder der grundsätzliche Durchsatz der zentrale Differenzierungsfaktor, sondern vor allem die Latenzverteilung.

Forschungsfrage F2: Hochlast- und Grenzlastverhalten.

Im replizierten Hochlastbereich veränderte sich das Verhalten beider Systeme deutlich gegenüber der Referenzreihe. Bei 500 simulierten Usern erreichte PostgreSQL mit durchschnittlich 632,35 Requests pro Sekunde einen höheren Durchsatz als MongoDB mit 559,46 Requests pro Sekunde. Auch die Median-Latenz lag bei PostgreSQL mit 220 ms niedriger als bei MongoDB mit 390 ms. Gleichzeitig zeigte die p99-Latenz bereits, dass die Bewertung nicht allein über den Median erfolgen kann: PostgreSQL lag bei 3.700 ms, MongoDB bei 2.766,67 ms.

Bei 1.000 simulierten Usern verschob sich das Bild. PostgreSQL fiel auf durchschnittlich 393,39 Requests pro Sekunde zurück, während MongoDB 513,11 Requests pro Sekunde erreichte. Auch die Tail-Latenzen stiegen bei PostgreSQL stärker an. Die p95-Latenz lag bei PostgreSQL bei 8.300 ms und die p99-Latenz bei 17.000 ms, während MongoDB bei 4.600 ms beziehungsweise 7.700 ms lag. Damit zeigt der replizierte Hochlastbereich, dass PostgreSQL bei 500 Usern günstigere Durchsatz- und Medianwerte erreichte, bei 1.000 Usern jedoch stärker in einen Sättigungs- beziehungsweise Degradationsbereich überging. MongoDB zeigte bei 1.000 Usern günstigere Durchsatz- und Tail-Latenzwerte, allerdings mit einer sehr niedrigen, aber messbaren Fehlerrate.

Die explorativen Grenzlastläufe bei 2.500 und 5.000 simulierten Usern ergänzen dieses Bild als bewusst nicht vollständig replizierte Grenzbereichsbetrachtung. PostgreSQL blieb in diesen Läufen ohne endgültige Failures, erreichte jedoch nur noch einen sehr niedrigen Durchsatz und sehr hohe Tail-Latenzen. Damit wurde die Degradation vor allem über Wartezeiten und Verarbeitungsgeschwindigkeit sichtbar. MongoDB erreichte in der stabilen Zieluserphase der Grenzlast zunächst höhere Durchsatzwerte und niedrigere p99-Latenzen als PostgreSQL, zeigte bei steigender Last jedoch ebenfalls deutliche Latenzdegradation und einzelne endgültige Checkout-Fehler. Endgültige Fehler wurden in den ausgewerteten Messreihen erstmals in der MongoDB-Hochlastreihe sichtbar und traten in der explorativen Grenzlast erneut auf; bei PostgreSQL wurden in den ausgewerteten Läufen keine endgültigen Failures gemessen.

Zusammenfassend zeigt Forschungsfrage F2, dass beide Systeme bei steigender paralleler Last deutliche Degradationseffekte aufweisen, diese jedoch unterschiedlich sichtbar werden. PostgreSQL zeigte im untersuchten Aufbau eine formal fehlerfreie, aber im Grenzbereich stark verlangsamte Verarbeitung. MongoDB zeigte im replizierten Hochlastbereich bei 1.000 Usern und insbesondere bei 2.500 simulierten Usern in der explorativen Grenzlast günstigere Durchsatz- und Tail-Latenzwerte, erreichte im Hoch- und Grenzlastbereich jedoch eine sehr niedrige, aber messbare Fehlerrate. Die Ergebnisse verdeutlichen damit unterschiedliche Degradations- und Stabilitätsmuster innerhalb des dokumentierten Versuchsaufbaus.

Forschungsfrage F3: Datenmengen-Sensitivität.

Die Datenmengen-Sensitivitätsanalyse untersuchte, wie sich eine Vergrößerung des Produktkatalogs bei konstanter Last auf beide Systeme auswirkt. Dafür wurde das Profil `medium` mit 50.000 Produkten dem Profil `large_catalog` mit 200.000 Produkten gegenübergestellt. Die Zahl der simulierten User blieb mit 500 Usern konstant, und beide Systeme wurden jeweils mit drei Wiederholungen ausgewertet. Diese Messreihe fokussiert damit gezielt den Einfluss eines größeren Produktkatalogs innerhalb des dokumentierten Versuchsaufbaus.

Bei PostgreSQL war der größere Produktkatalog mit moderaten Veränderungen der Leistungskennzahlen verbunden. Der mittlere Durchsatz sank von 632,35 auf 602,97 Requests pro Sekunde. Die Median-Latenz stieg von 220 ms auf 246,67 ms, die p95-Latenz von 1.466,67 ms auf 1.600 ms und die p99-Latenz von 3.700 ms auf 3.800 ms. Damit blieb PostgreSQL im untersuchten Sensitivitätsszenario auf einem ähnlichen Leistungsniveau wie im Profil `medium`.

Bei MongoDB fiel der Datenmengeneffekt deutlich stärker aus. Der mittlere Durchsatz sank von 559,46 auf 293,26 Requests pro Sekunde. Gleichzeitig stieg die Median-Latenz von 390 ms auf 930 ms, die p95-Latenz von 1.500 ms auf 3.933,33 ms und die p99-Latenz von 2.766,67 ms auf 6.533,33 ms. Die Fehlerrate blieb bei beiden Systemen in dieser Messreihe bei 0 %, sodass die Unterschiede vor allem über Durchsatz und Latenz sichtbar wurden.

Forschungsfrage F3 lässt sich daher so beantworten, dass die Vergrößerung des Produktkatalogs im dokumentierten Versuchsaufbau bei PostgreSQL mit moderaten Veränderungen der Leistungskennzahlen verbunden war, während MongoDB deutlich stärkere Veränderungen bei Durchsatz und Latenz zeigte. Diese Einordnung gilt für das konkrete Datenmodell, die verwendeten Indexe, das definierte Lastprofil und die lokale Testumgebung.

Forschungsfrage F4: Einordnung der beobachteten Unterschiede.

Die beobachteten Unterschiede lassen sich im dokumentierten Versuchsaufbau als szenariospezifische Leistungs- und Stabilitätsmuster einordnen. Beide Systeme führten dieselben fachlichen E-Commerce-Operationen aus, setzten diese jedoch gemäß ihres jeweiligen Datenmodells unterschiedlich um. PostgreSQL nutzte ein relationales Schema mit getrennten Tabellen, Fremdschlüsseln, Joins sowie separaten Bestands- und Warenkorbtabellen. MongoDB nutzte ein dokumentenorientiertes Modell, bei dem Bestandsdaten, Warenkorbpositionen und Bestellpositionen teilweise eingebettet gespeichert wurden.

Diese Modellierungsunterschiede beeinflussen, wie einzelne Operationen unter Last ausgeführt werden. Produkt- und Warenkorbabfragen können je nach Datenmodell unterschiedliche Zugriffspfade nutzen. Checkout-nahe Vorgänge beanspruchen zusätzlich Transaktions- und Schreibmechanismen, weil Bestellungen erzeugt, Bestände reduziert und Warenkörbe abgeschlossen werden. Die Messergebnisse beschreiben daher nicht nur Unterschiede zwischen zwei Datenbanksystemen, sondern auch Unterschiede zwischen zwei konkreten Modellierungs- und Umsetzungsvarianten desselben fachlichen Lastprofils.

Für die Bewertung ist außerdem relevant, dass die Untersuchung direkt auf der Datenbankebene durchgeführt wurde. Dadurch treten datenbanknahe Effekte des Lastprofils deutlicher hervor, weil Anwendungsschicht, HTTP-Kommunikation, Caching, externe Dienste und produktionsnahe Middleware nicht als überlagernde Einflussfaktoren wirken. Die Aussagekraft der Ergebnisse liegt entsprechend in der kontrollierten Einordnung dieses dokumentierten Datenbankversuchs.

Forschungsfrage F4 lässt sich daher so beantworten, dass die beobachteten Unterschiede als Ergebnis des konkreten Zusammenspiels aus Lastprofil, Datenmodellierung, Indexierung, Transaktionslogik, Clientstrategie und lokaler Testumgebung zu verstehen sind. PostgreSQL und MongoDB reagierten unter demselben fachlichen Lastprofil unterschiedlich auf steigende Parallelität und eine größere Produktdatenmenge. Aussagekräftig sind die Befunde vor allem für den hier dokumentierten Aufbau, die gewählten Datenmodelle, die verwendeten Indexe, die Clientstrategien und das definierte E-Commerce-nahe Lastprofil.

4.2. Methodische Einordnung und Limitationen

Die Aussagekraft der Ergebnisse ergibt sich aus dem kontrollierten Versuchsaufbau und ist an dessen dokumentierte Rahmenbedingungen gebunden. Die Untersuchung wurde in einer lokalen Docker-basierten Einzelknoten-Testumgebung durchgeführt. Beide Datenbanksysteme wurden dadurch unter einheitlich begrenzten Ressourcen betrachtet. Diese Festlegung unterstützt die Vergleichbarkeit und Reproduzierbarkeit der Messungen. Zugleich ist bei der Einordnung zu berücksichtigen, dass zusätzliche Hardware-Ressourcen, verteilte Datenbankkonfigurationen, Cloud-Umgebungen oder andere Betriebssystem- und Docker-Konfigurationen die beobachteten Grenzbereiche verschieben könnten.

Die direkte Messung auf der Datenbankebene ist ein weiterer zentraler Rahmenfaktor der Untersuchung. Die Lasttests greifen direkt auf PostgreSQL beziehungsweise MongoDB zu. Dadurch werden datenbanknahe Effekte des Lastprofils klarer sichtbar, weil Anwendungsschicht, HTTP-Kommunikation, Frontend, Caching, Authentifizierung und externe Dienste keine zusätzlichen Einflussfaktoren bilden. Die Ergebnisse beschreiben damit gezielt das Verhalten der Datenbanksysteme innerhalb eines isolierten Datenbankvergleichs; eine vollständige End-to-End-Betrachtung eines produktiven Online-Shops wäre ein eigenständiger Untersuchungsgegenstand.

Auch das Lastprofil und die Datengrundlage bestimmen die Reichweite der Ergebnisse. Das Lastprofil bildet typische datenbanknahe E-Commerce-Vorgänge ab, etwa Produktabfragen, Warenkorboperationen und checkoutnahe Transaktionen. Die Gewichtung dieser Operationen ist eine modellierte Annahme, die typische datenbanknahe Nutzungsmuster eines Online-Shops abbildet, jedoch nicht aus realen Zugriffsdaten eines konkreten Shops kalibriert wurde. Die synthetisch erzeugten Datensätze ermöglichen eine reproduzierbare und für beide Systeme gleiche fachliche Ausgangsbasis. Reale Produktdaten, saisonale Zugriffsmuster, unterschiedliche Warenkorbgrößen oder nutzerspezifische Kaufpfade könnten zu anderen Lastverteilungen und damit zu anderen Messergebnissen führen.

Die Ergebnisse sind außerdem an die konkrete Modellierung der beiden Systeme gebunden. PostgreSQL wurde mit einem relationalen Schema, getrennten Tabellen, Fremdschlüsseln und definierten Indexen untersucht. MongoDB wurde mit einem dokumentenorientierten Modell untersucht, in dem zusammengehörige Daten wie Bestandsinformationen, Warenkorbpositionen und Bestellpositionen teilweise eingebettet wurden. Die gemessenen Unterschiede spiegeln daher nicht nur die Datenbanksysteme selbst wider, sondern auch die gewählten Datenmodelle, Indexe, Transaktionsabläufe und Clientstrategien.

Die Wiederholungsstruktur der Messreihen ist ebenfalls zu berücksichtigen. Die Referenzreihe, der replizierte Hochlastbereich und die Datenmengen-Sensitivitätsanalyse wurden mit jeweils drei Wiederholungen pro relevanter Konfiguration ausgewertet. Dadurch lassen sich dort Mittelwerte und Streuungsmaße betrachten. Die explorativen Grenzlastläufe bei 2.500 und 5.000 simulierten Usern wurden dagegen als Einzelmessungen durchgeführt. Sie zeigen das beobachtete Verhalten der Systeme im stark belasteten Grenzbereich, werden jedoch als explorative Einzelmessungen ohne Streuungsaussage interpretiert.

Schließlich wurden beide Systeme in festgelegten Konfigurationen untersucht. PostgreSQL wurde in den Hochlast-, Grenzlast- und Sensitivitätsläufen mit erhöhter Verbindungsgrenze betrieben; MongoDB wurde in der final ausgewerteten Hochlast-, Grenzlast- und Sensitivitätsreihe mit gemeinsamem Client und definiertem Connection Pool verwendet. Diese Festlegung unterstützt einen klar abgegrenzten Vergleich der dokumentierten Konfigurationen. Gezielte Optimierungen, etwa produktionsnahes Connection Pooling, alternative Speicherparameter, zusätzliche Indexvarianten oder Query-Anpassungen, könnten die beobachteten Leistungs- und Degradationsmuster verändern beziehungsweise in andere Lastbereiche verschieben.

4.3. Ausblick

Aus den Ergebnissen und der methodischen Einordnung der Arbeit ergeben sich mehrere Ansatzpunkte für weiterführende Untersuchungen. Ein erster naheliegender Schritt wäre die Erweiterung des Versuchsaufbaus zu einer End-to-End-Testumgebung. In der vorliegenden Arbeit wurde die Datenbankebene gezielt isoliert betrachtet, um datenbanknahe Effekte des Lastprofils kontrolliert sichtbar zu machen. Eine weiterführende Untersuchung könnte die Datenbanksysteme über eine Anwendungsschicht mit HTTP-API, Geschäftslogik, Serialisierung, Authentifizierung und gegebenenfalls Caching ansprechen. Dadurch ließe sich prüfen, wie stark die beobachteten Durchsatz-, Latenz- und Fehlermuster auch in einem vollständigen Anwendungssystem sichtbar bleiben oder durch andere Systemschichten überlagert werden.

Ein zweiter Ansatzpunkt liegt in realitätsnäheren und variableren Lastprofilen. Das in dieser Arbeit verwendete Lastprofil bildet zentrale datenbanknahe E-Commerce-Vorgänge modellhaft und reproduzierbar ab. Weiterführende Arbeiten könnten reale oder realitätsnah rekonstruierte Zugriffsdaten einbeziehen, etwa Sitzungsverläufe, Suchmuster, Warenkorbgrößen, Checkout-Raten, zeitabhängige Lastspitzen oder veränderte Mischungsverhältnisse zwischen lesenden und schreibenden Operationen. Dadurch ließe sich untersuchen, wie empfindlich die Ergebnisse auf andere Gewichtungen, Operationen und Abfolgen reagieren.

Darüber hinaus bieten die beobachteten Hochlast-, Grenzlast- und Sensitivitätsbefunde Ansatzpunkte für gezielte Optimierungsstudien. Bei PostgreSQL wäre insbesondere zu untersuchen, wie sich produktionsnahes Connection Pooling, alternative Verbindungsstrategien, geänderte Speicherparameter, zusätzliche Indexvarianten oder Query-Anpassungen auf den stark verlangsamten Grenzbereich und auf größere Produktkataloge auswirken. Bei MongoDB könnten unterschiedliche Dokumentmodellierungen, alternative Einbettungs- beziehungsweise Referenzierungsstrategien, weitere Indexkonfigurationen, angepasste Pooling-Parameter und veränderte Transaktionsabläufe betrachtet werden. Solche Untersuchungen könnten zeigen, ob sich die in dieser Arbeit beobachteten Degradations- und Sensitivitätsmuster reduzieren oder in andere Last- beziehungsweise Datenmengenbereiche verschieben lassen.

Auch die Infrastruktur bietet Erweiterungspotenzial. Die vorliegende Untersuchung basiert auf einer lokalen Einzelknoten-Umgebung mit festen Ressourcenlimits. Zukünftige Arbeiten könnten verteilte Datenbankkonfigurationen, Cloud-basierte Umgebungen, Replikation, Sharding oder mehrere getrennte Lastgeneratoren einbeziehen. Dadurch ließe sich untersuchen, ob die beobachteten Leistungs- und Stabilitätsmuster auch unter stärker produktionsnahen Infrastrukturbedingungen auftreten oder sich durch horizontale Skalierung verändern.

Ein weiterer möglicher Anschluss besteht in einer tieferen Analyse einzelner versuchsrelevanter Implementierungsentscheidungen. Besonders relevant wären dabei die konkrete Umsetzung der sechs Lastprofiloperationen, die checkoutnahen Transaktionsabläufe, die MongoDB-Retry-Logik, die Wahl der Client- beziehungsweise Connection-Pooling-Strategien sowie die Datengenerierung für die Profile `medium` und `large_catalog`. Eine solche Analyse könnte genauer herausarbeiten, welche Bestandteile des experimentellen Aufbaus die gemessenen Leistungs-, Sensitivitäts- und Fehlermuster besonders beeinflussen.

Schließlich ließe sich der Systemvergleich erweitern. Neben PostgreSQL und MongoDB könnten weitere relationale, dokumentenorientierte oder NewSQL-Systeme unter demselben E-Commerce-nahen Lastprofil betrachtet werden. Dadurch könnte geprüft werden, welche der beobachteten Effekte systemspezifisch sind und welche stärker aus Datenmodellierung, Lastprofil, Transaktionsgestaltung oder Versuchsumgebung entstehen.

4.4. Fazit

Die Arbeit hat gezeigt, dass Datenbankleistung unter hoher Last im E-Commerce nicht allein über einzelne Kennzahlen wie maximalen Durchsatz bewertet werden sollte. Aussagekräftig wird die Bewertung erst durch die gemeinsame Betrachtung von Durchsatz, Latenzverteilung, Fehlerrate, Datenmodell, Lastprofil und Versuchsumgebung. Im kontrollierten Referenzbereich arbeiteten PostgreSQL und MongoDB fehlerfrei und mit ähnlichem Durchsatz. Unter höherer paralleler Last, im explorativen Grenzbereich und bei vergrößertem Produktkatalog traten jedoch deutlich unterschiedliche Leistungs- und Stabilitätsmuster hervor.

Damit beantwortet die Arbeit die zentrale Problemstellung durch eine differenzierte Einordnung der untersuchten Systeme unter den dokumentierten Versuchsbedingungen. PostgreSQL zeigte im untersuchten Aufbau besonders im Referenzbereich, bei 500 Usern und in der Datenmengen-Sensitivitätsanalyse günstige Leistungswerte mit niedrigen Median-Latenzen und nur moderaten Auswirkungen des größeren Produktkatalogs. MongoDB zeigte im replizierten Hochlastbereich bei 1.000 Usern sowie insbesondere bei 2.500 Usern in der explorativen Grenzlast günstige Durchsatz- und Tail-Latenzwerte, erreichte im Hoch- und Grenzlastbereich jedoch einzelne endgültige Checkout-Fehler. Die Ergebnisse verdeutlichen damit, dass die Bewertung stark vom konkreten Lastprofil, der Modellierung, der Clientstrategie und der gewählten Systemkonfiguration abhängt.

Der Beitrag der Arbeit liegt in einem nachvollziehbaren, reproduzierbaren und E-Commerce-nahen Vergleich zweier konkreter Datenbanksysteme unter kontrollierten Bedingungen. Die Untersuchung schafft damit eine Grundlage, um Leistungsdegradation, Stabilität und Datenmengen-Sensitivität nicht abstrakt, sondern anhand eines dokumentierten experimentellen Aufbaus zu betrachten. Die zentrale Erkenntnis besteht darin, dass PostgreSQL und MongoDB im untersuchten Szenario keine einheitliche Leistungsrangfolge bilden, sondern je nach Lastbereich und Datenmengenszenario unterschiedliche Stärken, Degradationsverläufe und Stabilitätsmuster zeigen.

A. Ergänzende Versuchsdokumentation

Dieser Anhang gibt einen Überblick über die ergänzenden Materialien, die zur Nachvollziehbarkeit und Reproduzierbarkeit des empirischen Versuchsaufbaus bereitgestellt werden. Sie umfassen insbesondere die standardisierte Versuchspipeline, die Konfiguration der Testumgebung, den Quellcode zur Datengenerierung, die Lasttest-Skripte, die Rohdaten der Locust-Messläufe, die aggregierten Ergebnisdateien sowie die erzeugten Diagramme. Vollständige Quellcode- und Ergebnisdateien werden nicht im Fließtext abgedruckt, sondern im digitalen Begleitordner bereitgestellt.

A.1. Standardisierte Versuchspipeline

Die standardisierte Versuchspipeline dokumentiert den Ablauf von der Auswahl der Versuchsreihe über Datengenerierung, Seeding und Lasttestdurchführung bis zur Validierung und Ergebnisaggregation. Sie ergänzt die Beschreibung des Vergleichsdesigns in Abschnitt 2.3 und zeigt die übergreifende Reihenfolge der wesentlichen Arbeitsschritte.

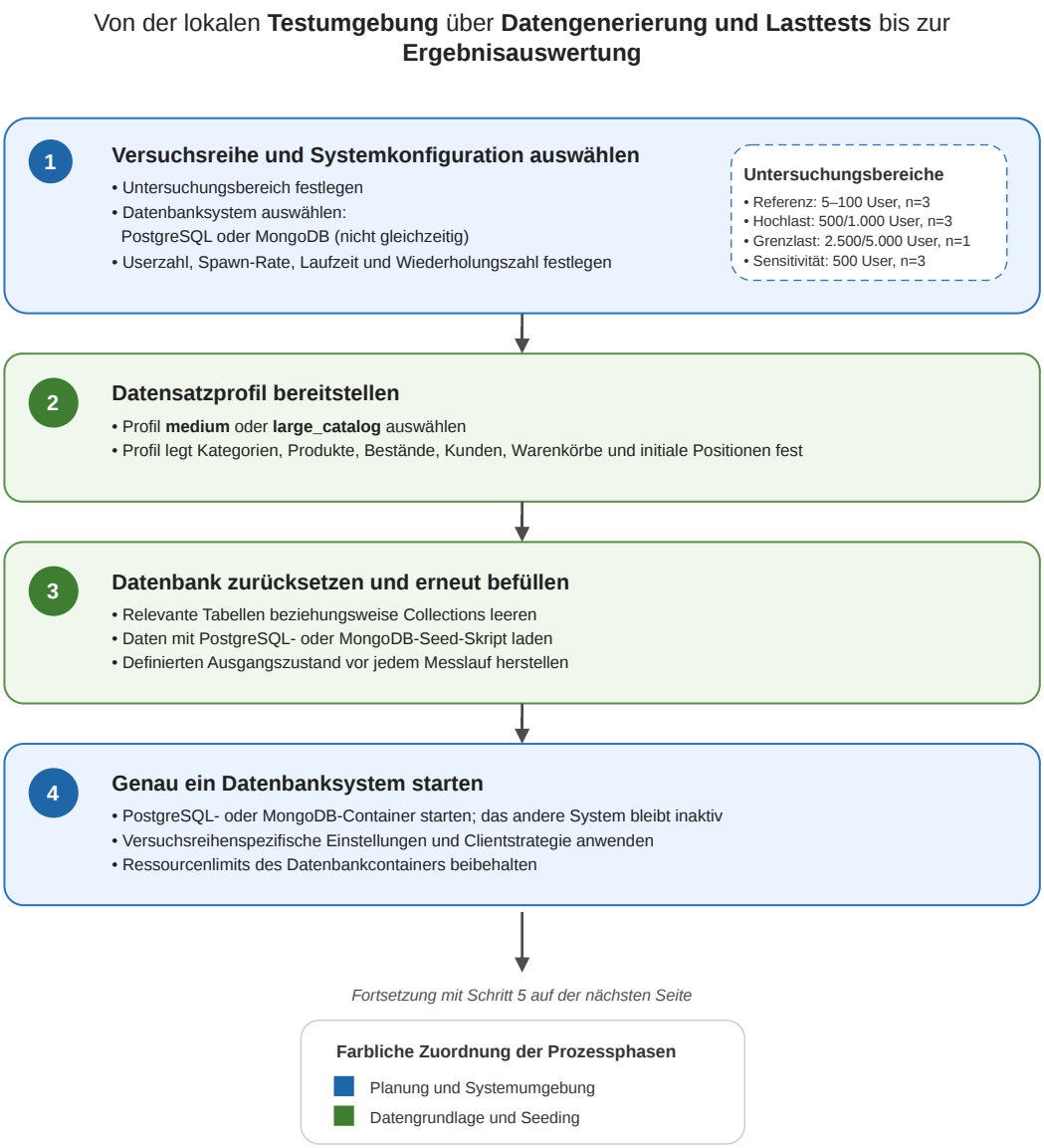


Abbildung A.1.: Standardisierte Versuchspipeline, Teil 1: Vorbereitung, Datengrundlage und Systemauswahl

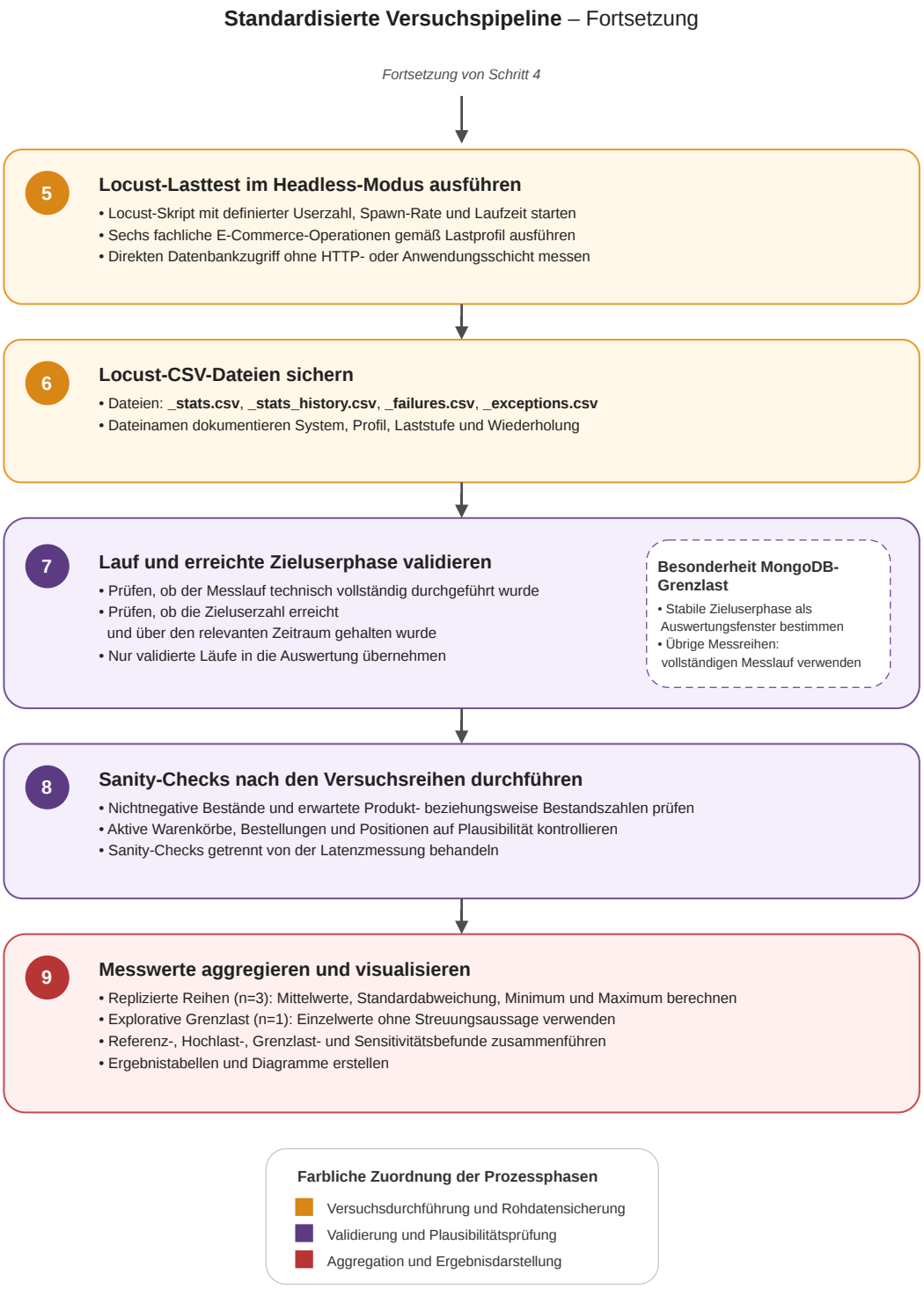


Abbildung A.2.: Standardisierte Versuchspipeline, Teil 2: Durchführung, Validierung und Auswertung

A.2. Digitale Begleitmaterialien

Tabelle A.1 fasst die wichtigsten digitalen Begleitmaterialien der empirischen Untersuchung zusammen. Die Ordnerangaben beziehen sich auf die Struktur des digitalen Begleitordners.

Die vollständigen Rohdaten der einzelnen Messläufe werden nicht im gedruckten Anhang dargestellt, da sie für den Fließtext zu umfangreich sind. Stattdessen werden sie im digitalen Begleitordner bereitgestellt. Die schriftliche Auswertung in Kapitel 3 basiert auf den validierten Rohdaten, den aggregierten Ergebnisdateien und den daraus erzeugten Diagrammen.

Tabelle A.1.: Übersicht der ergänzenden Versuchsdokumentation

Material	**Beschreibung**
Messplan und Protokoll	Enthält den finalen Messplan, die Versuchskonfigurationen und ergänzende Protokolldateien zur Dokumentation der Messdurchführung. Die Dateien befinden sich im digitalen Begleitordner insbesondere unter `messplan/` beziehungsweise in den zugehörigen Dokumentationsdateien.
Versuchspipeline	Enthält die grafische Prozessdarstellung der empirischen Untersuchung. Die im Anhang eingebundene PDF-Datei liegt im LaTeX-Projekt unter `img/versuchspipeline_zweiseitig.pdf`.
Docker-Konfiguration	Enthält die Docker-Compose-Konfiguration der lokalen Testumgebung für PostgreSQL und MongoDB einschließlich Container-Images, Ports, Ressourcenlimits und zentraler Startparameter. Die Datei befindet sich unter `docker/`.
Datenmodell und Initialisierung	Enthält das PostgreSQL-Schema sowie das MongoDB-Initialisierungsskript mit den für das Lastprofil verwendeten Tabellen, Collections, Constraints und Indexen. Die Dateien befinden sich unter `postgres/` beziehungsweise `mongodb/`.
Datengenerierung und Seeding	Enthält die Skripte zur Erzeugung der synthetischen CSV-Datengrundlage sowie die Seed-Skripte zum Laden der Daten in PostgreSQL und MongoDB. Die Dateien befinden sich unter `scripts/`.
Lastprofile	Enthält die Locust-Lastprofile für PostgreSQL und MongoDB. Die Skripte bilden die datenbanknahen E-Commerce-Operationen ab und definieren die verwendeten Task-Gewichtungen. Die Dateien befinden sich unter `loadgen/`.
Rohdaten der Messläufe	Enthält die von Locust exportierten CSV-Dateien der Messläufe, insbesondere Statistik-, Verlaufs-, Failure- und Exception-Dateien. Die Dateien befinden sich je nach Versuchsreihe unter anderem unter `results/final/`, `results/highload/` und `results/large_catalog/`.
Aggregierte Ergebnisdateien	Enthält die aus den Rohdaten erzeugten zusammengeführten und aggregierten Ergebnisdateien. Dazu gehören insbesondere die Übersichten der Referenzreihe, der ergänzenden Hoch- und Grenzlastreihe sowie der Datenmengen-Sensitivitätsanalyse. Die Dateien befinden sich in den jeweiligen `summary/`-Ordnern der Ergebnisverzeichnisse.
Diagramme	Enthält die erzeugten Diagramme zu Durchsatz, Median-Latenz, p95-Latenz, p99-Latenz, Fehlerrate sowie zur Datenmengen-Sensitivität. Die Dateien befinden sich in den jeweiligen `plots/`-Ordnern der Ergebnisverzeichnisse.

Erklärung zur Abschlussarbeit

Hiermit versichere ich, die eingereichte Abschlussarbeit selbständig verfasst und keine andere als die von mir angegebenen Quellen und Hilfsmittel benutzt zu haben. Wörtlich oder inhaltlich verwendete Quellen wurden entsprechend den anerkannten Regeln wissenschaftlichen Arbeitens zitiert. Ich erkläre weiterhin, dass die vorliegende Arbeit noch nicht anderweitig als Abschlussarbeit eingereicht wurde. Das Merkblatt zum Täuschungsverbot im Prüfungsverfahren der Hochschule Augsburg habe ich gelesen und zur Kenntnis genommen. Ich versichere, dass die von mir abgegebene Arbeit keinerlei Plagiate, Texte oder Bilder umfasst, die durch von mir beauftragte Dritte erstellt wurden.

Augsburg, den 22. Juni 2026



Anzilal Wahyuasmoro

Abbildungsverzeichnis

3.1. p95-Latenz je Operation bei 100 Usern 34
3.2. Durchsatz der ergänzenden Hoch- und Grenzlastreihe 36
3.3. Median-Latenz der ergänzenden Hoch- und Grenzlastreihe 37
3.4. p95-Latenz der ergänzenden Hoch- und Grenzlastreihe 38
3.5. p99-Latenz der ergänzenden Hoch- und Grenzlastreihe 38
3.6. Fehlerrate der ergänzenden Hoch- und Grenzlastreihe 39
3.7. Datenmengen-Sensitivität bei 500 Usern: Durchsatz 44
3.8. Datenmengen-Sensitivität bei 500 Usern: Median-Latenz 45
3.9. Datenmengen-Sensitivität bei 500 Usern: p95-Latenz 45
3.10. Datenmengen-Sensitivität bei 500 Usern: p99-Latenz 46

A.1. Standardisierte Versuchspipeline, Teil 1: Vorbereitung, Datengrundlage und Systemauswahl 57
A.2. Standardisierte Versuchspipeline, Teil 2: Durchführung, Validierung und Auswertung 58

Tabellenverzeichnis

1.1. Überblick über die untersuchten E-Commerce-nahen Operationen 3

2.1. Anforderungen an das E-Commerce-nahe Lastprofil 8
2.2. Methodische Kriterien des Datenbankvergleichs 11
2.3. Architekturbezogene Dimensionen der Ergebnisinterpretation 12
2.4. Untersuchungsbereiche des empirischen Vergleichs 14
2.5. Kompakte Versuchspipeline der empirischen Untersuchung 15
2.6. Lokale Hardware- und Softwareumgebung 15
2.7. Container- und Datenbankkonfiguration 16
2.8. Verbindungsstrategien der ausgewerteten Versuchsreihen 17
2.9. Datensatzprofile der empirischen Untersuchung 18
2.10. Operationen und Gewichtungen des E-Commerce-nahen Lastprofils 19
2.11. Auswertungsfenster der untersuchten Messreihen 21

3.1. Ausgewertete Messreihen der empirischen Evaluation 31
3.2. Aggregierte Gesamtergebnisse der Referenzreihe bei 5 bis 100 Usern 33
3.3. Aggregierte Gesamtergebnisse des replizierten Hochlastbereichs 35
3.4. Ergebnisse der explorativen Grenzlastläufe bei 2.500 und 5.000 Usern. PostgreSQL wurde auf Basis des vollständigen Messlaufs ausgewertet; MongoDB auf Basis der stabilen Zieluserphase. 41
3.5. Datenmengen-Sensitivität bei 500 Usern 43

A.1. Übersicht der ergänzenden Versuchsdokumentation 60

Literaturverzeichnis

[1] Adobe. *Adobe: Cyber Monday Hits Record $14.25 Billion in Online Spending with Over $1 Billion Driven by Buy Now Pay Later*. 2. Dez. 2025. URL: https://news.adobe.com/news/2025/12/adobe-cyber-monday-hits-record (besucht am 25. 05. 2026) (siehe S. 1).

[2] Salesforce. *Salesforce Releases Cyber Week and AI-Driven Sales*. 5. Dez. 2025. URL: https://www.salesforce.com/news/press-releases/2025/12/05/cyber-week-ai-agents-sales/ (besucht am 21. 06. 2026) (siehe S. 1).

[3] Google. *Measure ecommerce*. 2026. URL: https://developers.google.com/analytics/devguides/collection/ga4/ecommerce (besucht am 25. 05. 2026) (siehe S. 1, 6).

[4] Rodrigo Laigner, Zhexiang Zhang, Yijian Liu, Leonardo Freitas Gomes und Yongluan Zhou. „Online Marketplace: A Benchmark for Data Management in Microservices“. In: *Proceedings of the ACM on Management of Data* 3.1 (Feb. 2025), 3:1–3:26. DOI: 10.1145/3709653 (siehe S. 1, 4, 6).

[5] Dimitri Van Landuyt, Marie Levrau, Vincent Reniers und Wouter Joosen. „An E-Commerce Benchmark for Evaluating Performance Trade-Offs in Document Stores“. In: *Big Data Analytics and Knowledge Discovery*. Bd. 14912. Lecture Notes in Computer Science. Springer Nature, 18. Aug. 2024, S. 284–290. DOI: 10.1007/978-3-031-68323-7_23 (siehe S. 1, 4, 6).

[6] Silvia Botros und Jeremy Tinley. *High Performance MySQL: Proven Strategies for Operating at Scale*. 4. Aufl. Sebastopol, CA: O'Reilly Media, Nov. 2021 (siehe S. 2).

[7] Google. *Service Level Objectives*. 2026. URL: https://sre.google/sre-book/service-level-objectives/ (besucht am 25. 05. 2026) (siehe S. 2, 10).

[8] Google. *Implementing SLOs*. 2026. URL: https://sre.google/workbook/implementing-slos/ (besucht am 25. 05. 2026) (siehe S. 2, 10).

[9] Toni Taipalus. „Database Management System Performance Comparisons: A Systematic Literature Review“. In: *Journal of Systems and Software* 208 (Feb. 2024), S. 111872. DOI: 10.1016/j.jss.2023.111872 (siehe S. 2, 4, 8, 9, 22).

[10] Antonios Makris, Konstantinos Tserpes, Giannis Spiliopoulos, Dimitrios Zissis und Dimosthenis Anagnostopoulos. „MongoDB Vs PostgreSQL: A Comparative Study on Performance Aspects“. In: *GeoInformatica* 25 (Apr. 2021), S. 243–268. DOI: 10.1007/s10707-020-00407-w (siehe S. 2, 5, 8, 12).

[11] Redgate Software. *DB-Engines Ranking: Popularity Ranking of Database Management Systems*. Mai 2026. URL: https://db-engines.com/en/ranking (besucht am 25. 05. 2026) (siehe S. 3).

[12] Carnegie Mellon Database Group. *BenchBase*. 2026. URL: https://db.cs.cmu.edu/projects/benchbase/ (besucht am 25. 05. 2026) (siehe S. 4).

[13] Carnegie Mellon Database Group. *BenchBase: Multi-DBMS SQL Benchmarking Framework*. 2026. URL: https://github.com/cmu-db/benchbase (besucht am 25. 05. 2026) (siehe S. 4).

[14] Jeffrey Dean und Luiz André Barroso. „The Tail at Scale“. In: *Communications of the ACM* 56.2 (Feb. 2013), S. 74–80. DOI: 10.1145/2408776.2408794 (siehe S. 10, 20).

[15] MongoDB. *Data Modeling in MongoDB*. 2026. URL: https://www.mongodb.com/docs/manual/data-modeling/ (besucht am 25. 05. 2026) (siehe S. 13, 18, 26).

[16] PostgreSQL Global Development Group. *PostgreSQL Documentation: 13.1. Introduction*. 2026. URL: https://www.postgresql.org/docs/current/mvcc-intro.html (besucht am 25. 05. 2026) (siehe S. 13, 25).

[17] Apache Software Foundation. *Apache JMeter User's Manual: Component Reference*. 2026. URL: https://jmeter.apache.org/usermanual/component_reference.html (besucht am 20. 06. 2026) (siehe S. 16).

[18] Apache Software Foundation. *Apache JMeter User's Manual: Getting Started*. 2026. URL: https://jmeter.apache.org/usermanual/get-started.html (besucht am 20. 06. 2026) (siehe S. 16).

[19] Grafana Labs. *Grafana k6 Documentation: Protocols*. 2026. URL: https://grafana.com/docs/k6/latest/using-k6/protocols/ (besucht am 20. 06. 2026) (siehe S. 16).

[20] Locust. *Retrieve test statistics in CSV format*. 2026. URL: https://docs.locust.io/en/stable/retrieving-stats.html (besucht am 31. 05. 2026) (siehe S. 16, 20).

[21] PostgreSQL Global Development Group. *PostgreSQL Documentation: Chapter 11. Indexes*. 2026. URL: https://www.postgresql.org/docs/current/indexes.html (besucht am 31. 05. 2026) (siehe S. 24).

[22] PostgreSQL Global Development Group. *PostgreSQL Documentation: SELECT*. 2026. URL: https://www.postgresql.org/docs/current/sql-select.html (besucht am 25. 05. 2026) (siehe S. 25).

[23] MongoDB. *Embedded Data in Your MongoDB Schema*. 2026. URL: https://www.mongodb.com/docs/manual/data-modeling/embedding/ (besucht am 25. 05. 2026) (siehe S. 26).

[24] MongoDB. *Indexes*. 2026. URL: https://www.mongodb.com/docs/manual/indexes/ (besucht am 25. 05. 2026) (siehe S. 27).

[25] MongoDB. *Write Operation Performance*. 2026. URL: https://www.mongodb.com/docs/manual/core/write-performance/ (besucht am 25. 05. 2026) (siehe S. 27).

[26] MongoDB. *Transactions in Applications*. 2026. URL: https://www.mongodb.com/docs/manual/core/transactions-in-applications/ (besucht am 31. 05. 2026) (siehe S. 28).